Semirings for database provenance

by *Yiannis Hadjicharalambous*

Supervisors: *Ivan Di Liberti* (University of Gothenburg) &
Phokion Kolaitis (UC Santa Cruz)

Department of Philosophy, Linguistics, and Theory of Science
University of Gothenburg

Thesis for the Master's degree in Logic
30 credits

5 June 2026

Abstract

This thesis provides an accessible exposition of the semiring-based framework for database provenance. Starting from the foundational results for the positive relational algebra, we examine the use of commutative semirings and the role of polynomials as a universal representation of provenance.

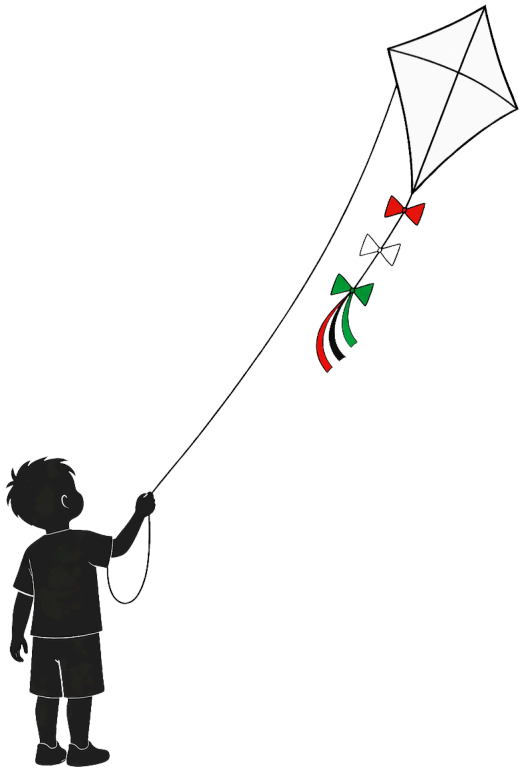
A central theme of this work is the extension of the framework to more expressive queries, specifically addressing the challenges posed by aggregation and difference. The study reviews existing literature, providing rigorous proofs for established results, and reformulating key concepts where further clarity is needed.

Acknowledgements

I would like to express my sincere appreciation to Ivan Di Liberti for giving me the opportunity to pursue the topics I find most interesting, and for his continuous support throughout this project. I am especially grateful that he connected me with Phokion Kolaitis, which led to the joint supervision of my master's thesis. I would also like to thank Phokion Kolaitis for agreeing to co-supervise my thesis despite his busy schedule, and for the attention and guidance he has generously provided. Of course, none of this would have been possible without the resources and community provided by the University of Gothenburg.

I am deeply grateful to the people I have met during my time in Gothenburg and value the time we have spent together. Their kindness and support have been a great source of encouragement throughout my studies. I will look back on these years with gratitude for everyone who made this journey so meaningful.

Finally, I want to express my heartfelt gratitude to my family. Thank you for your unwavering encouragement and for believing in my academic pursuits. I love you ως τον ουρανόν.



Contents

CHAPTER 1

Introduction

In database theory, provenance is a mechanism that relates data in an output to the contributing data in the input. Such information is important for many data management tasks and is increasingly relevant in a world where vast amounts of data are used in a variety of ways.

We will focus on provenance for data residing in a relational database. Database provenance is typically represented as a general form of annotation information, by tagging data on the tuple-level or value-level of a database relation.

Cui, Widom, and Wiener [Cui+00] were among the first to formalise a notion of provenance in the context of relational databases, called *lineage* (or *which-provenance*). They associated each tuple t present in the output of a query with a set of tuples present in the input, called the lineage of t. Intuitively, the lineage of t is meant to collect all of the input tuples that helped produce it.

from	to	
a	a	$\mathbf{p}$
a	b	$\mathbf{q}$
b	a	$\mathbf{r}$
b	c	$\mathbf{s}$

FIGURE 1. Edges relation

Consider the example relation shown in Figure 1 storing the edges (and their annotations in bold) of a graph and the following query (borrowed from [KG12]):

QUERY 1.0.1. Find all (x, y) such that y is reachable from x via a path of length 3.

In the output of this query we can find the tuple (a, a). One way of calculating this result tuple is by taking the tuple tagged with $\mathbf{p}$ three times. Another way is by taking the tuple $\mathbf{p}$, then the tuple $\mathbf{q}$, and then the tuple $\mathbf{r}$. Finally, we could take the tuple $\mathbf{q}$, then the tuple $\mathbf{r}$, and then the tuple $\mathbf{p}$. The lineage of (a, a) would then be $\{\mathbf{p}, \mathbf{q}, \mathbf{r}\}$.

While the notion of lineage defined by Cui et al. exposes a collection of input tuples that contribute to the creation of an output tuple t, it is not as precise as we might like. When an output tuple can be derived in multiple ways, lineage does not capture its individual witnesses but instead combines all contributing tuples together. This intuition was formalised by Buneman, Khanna, and Wang-Chiew [Bun+01] who introduced *why-provenance*, a notion that does capture the different witnesses of an output tuple. Continuing with the example of the *Edges* relation from Figure 1, the why-provenance of the output tuple (a, a) from our query would be $\{\{\mathbf{p}\}, \{\mathbf{p}, \mathbf{q}, \mathbf{r}\}\}$.

The why-provenance of an output tuple provides a set of witnesses for that tuple. However, it does not provide additional information on how the output tuple is actually derived. In our running example, why-provenance does not tell us that $\mathbf{p}$ is used three times in its first witness. It also does not tell us that there are two ways to derive $\{\mathbf{p}, \mathbf{q}, \mathbf{r}\}$ (up to associativity and commutativity).

To address these shortcomings, a seminal paper by Green, Karvounarakis, and Tannen [Gre+07] introduced *provenance semirings* (or *how-provenance*), in which tuples are

annotated with elements of a commutative semiring. This framework is built on the insight that data is essentially combined in two ways: alternatively and jointly. By considering these two modes of combination as algebraic operations, together with two constants to indicate absence and presence, we obtain an algebraic structure $(S, +, \cdot, 0, 1)$. Green et al. then generalise the relational algebra to operate on annotated tuples and identify the laws required for the standard relational algebra identities to remain valid, showing that these are precisely the axioms of a commutative semiring.

In this framework, the polynomial semiring $(\mathbb{N}[X], +, \cdot, 0, 1)$ is identified as the provenance domain, since it is the free commutative semiring. This means that it possesses the following universal property: for every semiring S and any function (valuation) $f : X \to S$, there exists a unique semiring homomorphism $h : \mathbb{N}[X] \to S$ such that $h(x) = f(x)$ for all $x \in X$.

Since the polynomial semiring enjoys this universal property, the variables in X can be substituted with elements of any commutative semiring, yielding a unique homomorphism into that semiring. In this way, different provenance models (as well as database semantics) arise as specialisations of the polynomial semiring.

This framework therefore allows for the evaluation of queries over any database by annotating source tuples with variables from an abstract set X. Under this approach, a relation is treated as an $\mathbb{N}[X]$-relation, where the provenance of each output tuple is expressed as a polynomial with natural number coefficients. These annotations propagate through query evaluation, where the semantics of the relational algebra operators are mapped directly to the addition and multiplication operations of the semiring. In the case of our example, the how-provenance of (a, a) is the polynomial $\mathbf{p}^3 + 2\mathbf{pqr}$, which gives more precise information than the why-provenance does.

1. Outline

While the paper by Green et al. was influential, it focused on the positive fragment of the relational algebra, where set difference is not included, and only briefly addressed the inclusion of recursion in the framework.

A significant body of further research has gone into extending the semiring framework to support more expressive queries. In what follows, we examine the results of the paper by Green et al. as well as subsequent extensions of the framework.

More specifically, Chapter 2 provides relevant background knowledge on database theory and abstract algebra. Chapter 3 then presents the semiring-based framework for provenance. We show how the notions of relation, database, and the relational algebra are generalised to accommodate annotations. Furthermore, we confirm that in order for standard relational algebra identities to continue to hold in this generalisation, the underlying structure of the annotations must be a commutative semiring. We present the fundamental theorem of the framework, stating that query evaluation commutes with semiring homomorphisms. We then end the chapter by showing that the polynomial semiring serves as the provenance domain and provide examples of notable semirings it generalises.

In Chapter 4, we investigate an extension of the semiring framework that facilitates the expression of aggregate queries. We show that semirings alone are insufficient for a uniform treatment of aggregation and consider the necessary algebraic modifications. We utilise the additional structure provided by semimodules and employ a tensor product construction as the provenance domain. We then define the operations of aggregation and grouping on this extended structure and conclude the chapter by demonstrating that the fundamental theorem continues to hold.

Chapter 5 provides a short survey of the various approaches used to define a difference operation in the semiring framework. Given that semirings lack additive inverses by definition, this presents a significant challenge. We classify these approaches into three main groups: the direct addition of algebraic identities to semirings, the use of a semiring's

natural order to define a monus operation, and the construction of quotient semirings, ultimately evaluating the various trade-offs that each approach entails.

Finally, Chapter 6 reviews topics not covered and discusses research challenges.

2. Contributions

This thesis provides an accessible exposition of semiring-based provenance, presenting the foundational theory, as well as important extensions. By combining intuitive explanation with technical rigour, the work translates specialised research into a structured narrative suitable for a graduate-level audience. The technical contributions of this work are as follows:

- Reformulated and proved the relationship between relational algebra identities and the commutative semiring axioms in Proposition 3.1.6.
- Provided proofs for the fundamental theorem (Theorem 3.1.7) and the universality of the provenance semiring (Corollary 3.2.1).
- Proved the insufficiency of standard semirings for modelling aggregation (Proposition 4.0.2).
- Provided a proof for the fundamental theorem in the context of simple aggregation (Theorem 4.1.7).
- Proved the compatibility conditions established in Theorem 4.1.12 and Theorem 4.1.13.
- Constructed the compatible semiring $S\mathbb{N}$ by generalising the congruence relations found in [Ams+11b].
- Proved that $S^M \cong S$ provided that $S \otimes M \cong M$ (Proposition 4.2.4).
- Reformulated the lifting of δ-semiring homomorphisms on $S^M \otimes M$ and proved the fundamental theorem for arbitrary aggregation (Theorem 4.2.6).
- Proved the fundamental theorem for semiring semantics in first-order logic, as developed by Grädel and Tannen (Theorem 5.2.3).

CHAPTER 2

Background

In this chapter, we review the theoretical background required for understanding the next chapters. Familiarity with general notions from logic is assumed.

1. Database theory

Although several database models exist, relational databases remain the most widely used and studied. The relational data model was introduced and popularised by Edgar Codd through a series of influential papers published in the early 1970s [Cod70; Cod71; Cod72]. Henceforth, we use the term database to mean a relational database. A database consists of finitely many relations (tables). Each relation has a finite number of attributes (columns) and tuples (rows).

More formally, we fix a countably infinite set of constants (values) $\mathbb{D}$, called the underlying *domain*. We also assume that a totally ordered countably infinite set $\mathcal{U}$ of *attribute names* is fixed. We use the symbols U, V for finite subsets of $\mathcal{U}$. We define a *tuple* as a function $t : U \to \mathbb{D}$, in which case we say that t is over the attribute set U.

It is sometimes helpful to instead view a tuple as an ordered n-tuple of constants, i.e. an element of the Cartesian product $\mathbb{D}^n$, where $n = |U|$. These two viewpoints are referred to as the *named* and *unnamed* perspectives, respectively.

Definition 2.1.1 (Relation). An n-ary *relation* R is a finite set of tuples over U or, equivalently, a finite subset of the Cartesian product $\mathbb{D}^n$, depending on the chosen perspective. A *database* D is a finite set of relations. ▽

Having formalised the representation of our data, we now turn to the formalisation of querying. Informally, a query specifies how new relations can be derived from an existing database. Formally, a *query* is a mapping that takes a database as input and returns a relation of fixed arity. We shall study several database query formalisms, including the relational algebra, the relational calculus, and Datalog.

The relational algebra and the relational calculus form the foundational query formalisms of the relational model. Relational algebra is an algebraic formalism in which queries are expressed by applying a sequence of operations to relations, while the relational calculus is a logical formalism in which queries are expressed as formulae of first-order logic. Despite their syntactic differences, Codd proved that the relational algebra and the relational calculus are equivalent in expressive power [Cod72]. Datalog was introduced a few years later as a logic programming language but has since found use in database theory as a recursive query language [CH85]. In what follows, we explore fragments of these formalisms in tandem, organised by increasing levels of expressive power.

Definition 2.1.2 (Conjunctive query). A first-order formula over a relational vocabulary is called a *conjunctive query* if it is built from atomic formulae using only conjunction and existential quantification. ▽

Example 2.1.3. The following conjunctive query: $\exists z_1 \exists z_2 \; E(x, z_1) \wedge E(z_1, z_2) \wedge E(z_2, y)$ returns all pairs (x, y) such that there is a path of length 3 between x and y in a graph with edge relation E. ▽

Remark. Note that the definition of a conjunctive query imposes no requirement on the formula to be in prenex normal form. Also note that a conjunctive query can be a sentence (a formula with no free variables).

As hinted above, relational algebra defines operations that transform one or more input relations to return an output relation. We now consider two such formulations that correspond to the named and unnamed perspectives. They are equivalent in expressive power to each other and to the conjunctive queries.

Definition 2.1.4 (SPJR algebra). Let R and S be relations over attribute sets U and V, respectively. The *SPJR algebra* consists of the following operations:

- Selection, $\sigma_\theta(R)$: Given an equality θ of the form $A = B$ or $A = a$ (where $A, B \in U$ and $a \in \mathbb{D}$), selection is defined as:

$$\sigma_\theta(R) = \begin{cases} \{t \in R \mid t(A) = t(B)\} & \text{if } \theta \text{ is of the form } A = B, \\ \{t \in R \mid t(A) = a\} & \text{otherwise.} \end{cases}$$

  where $t(A)$ denotes the value of the tuple t at attribute A.

- Projection, $\pi_L(R)$: Given a set of attributes $L \subseteq U$, projection is defined as:

$$\pi_L(R) = \{t[L] \mid t \in R\},$$

  where $t[L]$ denotes the tuple t restricted to the attribute set L.

- (Natural) join, $R \bowtie S$: Join combines tuples from R and S using any shared attributes:

$$R \bowtie S = \{t \text{ over } U \cup V \mid t[U] \in R \text{ and } t[V] \in S\}.$$

- Renaming, $\rho_\beta(R)$: Given a bijection $\beta : U \to U'$, renaming is defined as:

$$\rho_\beta(R) = \{t' \text{ over } U' \mid t' \circ \beta \in R\}.$$

▽

Definition 2.1.5 (SPC algebra). Let R and S be relations of arity n and m, respectively. The *SPC algebra* consists of the following operations:

- Selection, $\sigma_\theta(R)$: Given an equality θ of the form $i = j$ or $i = a$ (where $1 \leq i, j \leq n$ and $a \in \mathbb{D}$), selection is defined as:

$$\sigma_\theta(R) = \begin{cases} \{(a_1, \ldots, a_n) \in R \mid a_i = a_j\} & \text{if } \theta \text{ is of the form } i = j, \\ \{(a_1, \ldots, a_n) \in R \mid a_i = a\} & \text{otherwise.} \end{cases}$$

- Projection, $\pi_L(R)$: Given a sequence of indices $L = (i_1, i_2, \ldots, i_k)$ where each $i_j \in \{1, \ldots, n\}$, projection is defined as:

$$\pi_L(R) = \{(a_{i_1}, a_{i_2}, \ldots, a_{i_k}) \mid (a_1, a_2, \ldots, a_n) \in R\}.$$

  The resulting relation has arity $k = |L|$.

- Cartesian product, $R \times S$: The Cartesian product is the set of all concatenated tuples from R and S:

$$R \times S = \{(a_1, \ldots, a_n, b_1, \ldots, b_m) \mid (a_1, \ldots, a_n) \in R \text{ and } (b_1, \ldots, b_m) \in S\}.$$

  The resulting relation has arity $n + m$.

▽

Example 2.1.6. Recall Query 1.0.1, which we now formulate in relational algebra. The following expressions represent the query in SPJR and SPC, respectively:

$$\pi_{from,to}((\rho_{to\mapsto m}(Edges) \bowtie \rho_{from\mapsto m,\ to\mapsto n}(Edges)) \bowtie \rho_{from\mapsto n}(Edges))$$

$$\pi_{1,6}(\sigma_{2=3}(\sigma_{4=5}((Edges \times Edges) \times Edges)))$$

▽

We now extend Definition 2.1.4 and Definition 2.1.5 by adding a binary set union operation. For two relations R and S over the same attribute set (or, depending on the perspective, with the same arity):

$$R \cup S = \{t \mid t \in R \text{ or } t \in S\}.$$

This yields the *SPJRU* and *SPCU* algebras, respectively. Again, these two algebras are equivalent in expressive power. The resulting algebra is commonly referred to as the *positive relational algebra*.

When extending conjunctive queries, disjunction must be handled with care. Allowing unrestricted disjunction may leave free variables unconstrained, causing them to range over the entire domain and potentially return an infinite output. Unions of conjunctive queries (UCQs) avoid this problem by requiring each disjunct to be a conjunctive query over the same free variables.

Definition 2.1.7 (Union of conjunctive queries). A first-order formula over a relational vocabulary is called a *union of conjunctive queries* if it is of the form

$$\varphi_1(\vec{x}) \vee \varphi_2(\vec{x}) \vee \cdots \vee \varphi_k(\vec{x}),$$

where each $\varphi_i(\vec{x})$ is a conjunctive query and all φ_i have the same free variables $\vec{x}$. ▽

Theorem 2.1.8. *Unions of conjunctive queries and the positive relational algebra are equivalent in expressive power.*

We are now ready to define the relational algebra and the relational calculus. The *relational algebra* is the result of adding a binary set difference operation to the positive relational algebra. For two relations R and S over the same attribute set (or, depending on the perspective, with the same arity):

$$R - S = \{t \mid t \in R \text{ and } t \notin S\}.$$

Extending UCQs with negation is not as straightforward as it is for the relational algebra. Just as with extending conjunctive queries with union, care must be taken to ensure that the resulting calculus remains well-defined since unrestricted negation can lead to infinite results.

The most popular, as well as the most intuitive, approach for extending unions of conjunctive queries to the relational calculus is the active domain semantics, in which all variables are restricted to range over the set of all values appearing in the database.

Definition 2.1.9 (Active domain). The *active domain* of a database D is the set of all constants that occur in the relations of D, denoted by $adom(D)$. The *active domain semantics* of a first-order formula is defined such that all variables are restricted to range over $adom(D)$. ▽

The *relational calculus* is then defined as the set of all queries expressible in first-order logic over a relational vocabulary, evaluated under active domain semantics.

Theorem 2.1.10 (Codd's theorem [Cod72]). *The relational algebra and the relational calculus are equivalent in expressive power.*

This is an appropriate point to introduce Datalog. As mentioned earlier, Datalog is a logic programming language, but here we consider it as a query language. While its syntax is relatively minimal, it can express queries that the relational algebra and calculus cannot, namely recursive queries.

A Datalog program is a finite set of rules and facts. Rules are in the form of an implication where the body (antecedent) is a conjunctive query of atoms and the head (consequent) is an atom. Facts are rules that have no body. From a database-theoretic perspective, we can think of a Datalog program as a query against a database. We look at a classic example before giving the formal definition.

Example 2.1.11. This is a Datalog program defining the transitive closure of the Edge relation:

$$\text{Path}(x, y) \leftarrow \text{Edge}(x, y)$$
$$\text{Path}(x, y) \leftarrow \text{Edge}(x, z) \wedge \text{Path}(z, y)$$

Intuitively, the first rule is the base case: there is a path from x to y whenever there is a direct edge from x to y. The second rule is the recursive case: there is a path from x to y if x has an edge to some z and there is already a path from z to y. ▽

Definition 2.1.12 (Datalog program). A *Datalog rule* is an expression of the form

$$H \leftarrow B_1 \wedge \cdots \wedge B_n$$

where H and $B_1, \ldots, B_n$ are atoms. The atom H is called the *head* and $B_1 \wedge \cdots \wedge B_n$ is called the *body*. A rule with an empty body ($n = 0$) is called a *fact*. A finite set of Datalog rules is called a *Datalog program*. ▽

A crucial distinction is made between two kinds of atoms:

- Extensional database (EDB) predicates: relations that only occur in the body of a rule; these constitute the input of a Datalog program.
- Intensional database (IDB) predicates: relations that occur in the head of at least one rule; these are derived by the Datalog program.

A Datalog program is *recursive* if an IDB predicate occurs in the body and the head of a rule, not necessarily the same rule.

Datalog can be seen as a generalisation of UCQs, since non-recursive Datalog programs coincide in expressive power with UCQs, and hence with the positive relational algebra. However, while the UCQs are contained in Datalog, the relational calculus is not, since, for example, Datalog cannot express universal first-order sentences. At the same time, Datalog is not contained in the relational calculus, since transitive closure is not expressible in it. The following theorem describes the expressive power common to both Datalog and the relational calculus.

Theorem 2.1.13. *The queries expressible in both Datalog and the relational calculus are exactly the unions of conjunctive queries.*

Proof. This result follows from [Ros08, Theorem 1.7], combined with the fact that Datalog queries are preserved under homomorphisms and that existential-positive first-order formulas correspond to unions of conjunctive queries. □

The goal of this section has been to present the most relevant material from database theory. For a more rigorous exposition, the reader is referred to the standard reference by Abiteboul, Hull, and Vianu [Abi+95].

2. Algebra

This section introduces the algebraic structures used throughout this work. In particular, we review semirings and techniques used in the study of database provenance.

Definition 2.2.1 (Monoid). A *monoid* is a set S equipped with an associative binary operation $\star$, together with an identity element $e \in S$ such that for every $a \in S$, we have $e \star a = a \star e = a$. A monoid where $\star$ is commutative is called a *commutative monoid*. ▽

The identity element of a monoid is unique, since if both e_1 and e_2 satisfy the above properties, then $e_1 = e_1 \star e_2 = e_2$. A monoid can therefore be represented by the triple $(S, \star, e)$. More generally, when the structure is clear from context, we may refer to an algebraic structure simply by its underlying set.

Definition 2.2.2 (Group). A *group* is a monoid in which every element has an inverse. That is, for each $a \in S$, there exists $b \in S$ such that $a \star b = b \star a = e$. A group where $\star$ is commutative is called an *abelian group*. ▽

Definition 2.2.3 (Ring). A *ring* is a set S equipped with two binary operations $+$ (addition) and $\cdot$ (multiplication) such that $(S, +, 0)$ is an abelian group, $(S, \cdot, 1)$ is a monoid, and multiplication distributes over addition, meaning that for all $a, b, c \in S$:

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$
$$(b + c) \cdot a = (b \cdot a) + (c \cdot a)$$

A ring where multiplication is commutative is called a *commutative ring*. ▽

Since the additive and multiplicative identities are unique, a ring can be represented by the 5-tuple $(S, +, \cdot, 0, 1)$.

Definition 2.2.4 (Semiring). A *semiring* is a set S equipped with two binary operations $+$ (addition) and $\cdot$ (multiplication) such that $(S, +, 0)$ is a commutative monoid, $(S, \cdot, 1)$ is a monoid, multiplication distributes over addition, and for all $a \in S$, we have $0 \cdot a = a \cdot 0 = 0$. A semiring where multiplication is commutative is called a *commutative semiring*. ▽

Example 2.2.5. The integers $\mathbb{Z}$ with addition form an abelian group, $(\mathbb{Z}, +, 0)$, and with multiplication they form a commutative monoid, $(\mathbb{Z}, \cdot, 1)$. Taken together, the integers form a commutative ring, $(\mathbb{Z}, +, \cdot, 0, 1)$. The natural numbers $\mathbb{N}$ instead form a commutative semiring, $(\mathbb{N}, +, \cdot, 0, 1)$. ▽

Remark. Note that the distinction between a semiring and a ring is that a semiring does not require additive inverses.

We now review polynomial semirings, which will play a leading role in our development.

Definition 2.2.6 (Polynomial semiring). Let S be a commutative semiring and let X be a set of variables. The *polynomial semiring* $S[X]$ consists of expressions of the form

$$\sum_{m \in \mathcal{M}} s_m m,$$

where $\mathcal{M}$ is the set of monomials (finite products of variables) in X, and the coefficient s_m is in S. We require the set $\{m \in \mathcal{M} \mid s_m \neq 0\}$ to be finite, meaning that each element of $S[X]$ can be uniquely represented as a finite sum by omitting terms with zero coefficients.

Addition in $S[X]$ is defined by combining equal monomials and adding their coefficients in S, and multiplication is defined by distributivity, where the product of two coefficients is taken in S and the product of two monomials is found by adding the exponents of their corresponding variables. ▽

EXAMPLE 2.2.7. Let $\mathbb{N}$ be the semiring of natural numbers and let $X = \{x\}$ be a set with a single variable.

An example of an element in the polynomial semiring $\mathbb{N}[x]$ is:

$$p = 5 + 2x + 10x^3$$

This is a polynomial because it is a finite sum: the coefficients for x^0, x^1, and x^3 are nonzero, while the coefficients for all other powers of x are zero. Consider another polynomial $q \in \mathbb{N}[x]$ defined as:

$$q = 2 + 3x$$

Addition is performed by combining coefficients of like powers:

$$p + q = (5 + 2) + (2 + 3)x + 10x^3 = 7 + 5x + 10x^3$$

Multiplication is performed by distributing p over q:

$$\begin{aligned} p \cdot q &= (5 + 2x + 10x^3) \cdot (2 + 3x) \\ &= 5(2 + 3x) + 2x(2 + 3x) + 10x^3(2 + 3x) \\ &= (10 + 15x) + (4x + 6x^2) + (20x^3 + 30x^4) \\ &= 10 + 19x + 6x^2 + 20x^3 + 30x^4 \end{aligned}$$

▽

An important notion in the study of algebraic structures is that of a homomorphism: a map between algebraic structures of the same type that preserves the given operations. We now consider the formal definition of a homomorphism for semirings.

DEFINITION 2.2.8 (Semiring homomorphism). Let S and S' be two semirings. A *semiring homomorphism* is a function $h : S \to S'$ such that for all $x, y \in S$:

$$\begin{aligned} h(0_S) &= 0_{S'} & h(x +_S y) &= h(x) +_{S'} h(y), \\ h(1_S) &= 1_{S'} & h(x \cdot_S y) &= h(x) \cdot_{S'} h(y). \end{aligned}$$

▽

A few examples are in order.

EXAMPLE 2.2.9. Let $S = \mathbb{N}$ and $S' = \mathbb{N}[X]$, the polynomial semiring over a set of variables X. Define $h : \mathbb{N} \to \mathbb{N}[X]$:

$$n \mapsto n \cdot 1.$$

Then h is a semiring homomorphism because for all $m, n \in \mathbb{N}$:

$$\begin{aligned} h(m + n) &= (m + n) \cdot 1 = m \cdot 1 + n \cdot 1 = h(m) + h(n), \\ h(m \cdot n) &= (mn) \cdot 1 = (m \cdot 1) \cdot (n \cdot 1) = h(m) \cdot h(n), \end{aligned}$$

and clearly $h(0) = 0 \cdot 1 = 0$ and $h(1) = 1 \cdot 1 = 1$. ▽

EXAMPLE 2.2.10. Let $S = \mathbb{N}$ and $S' = \mathbb{B} = (\{0, 1\}, \vee, \wedge, 0, 1)$, the Boolean semiring. Define $h : \mathbb{N} \to \mathbb{B}$:

$$h(n) = \begin{cases} 0 & n = 0, \\ 1 & n > 0. \end{cases}$$

We verify that h is a semiring homomorphism. It clearly preserves the identity elements: $h(0) = 0$ and $h(1) = 1$. Also, for any $m, n \in \mathbb{N}$:

$$h(m + n) = \begin{cases} 0 & m + n = 0, \\ 1 & m + n > 0 \end{cases} = h(m) \vee h(n).$$

$$h(m \cdot n) = \begin{cases} 0 & m \cdot n = 0, \\ 1 & m \cdot n > 0 \end{cases} = h(m) \wedge h(n).$$

▽

Having defined semiring homomorphisms and provided some examples, we now turn to the concept of a free semiring. Intuitively, a free semiring on a set of generators X is characterised by the property that any function from X to any semiring S' extends uniquely to a semiring homomorphism from the free semiring to S'.

In other words, a free semiring provides a universal setting in which elements of X can be combined using addition and multiplication, without imposing any relations beyond those required by the semiring axioms.

Definition 2.2.11 (Free semiring). A semiring S is called a *free semiring* on a set of generators X if $X \subseteq S$ and for any semiring S' and any function $f : X \to S'$, there exists a unique semiring homomorphism $h : S \to S'$ such that $h(x) = f(x)$ for all $x \in X$. ▽

We can think of f as a valuation function that assigns each variable in X to an element of a semiring S'. The polynomial semiring $\mathbb{N}[X]$ fits the description of a free commutative semiring on generators X, so f extends uniquely to a semiring homomorphism $h : \mathbb{N}[X] \to S'$. That is, h evaluates a polynomial by replacing each variable with its image under f and performing all sums and products within S'.

Proposition 2.2.12. *The semiring $\mathbb{N}[X]$ is the free commutative semiring on the set X. That is, for every commutative semiring S and every function $f : X \to S$, there exists a unique semiring homomorphism $h : \mathbb{N}[X] \to S$ such that $h(x) = f(x)$ for all $x \in X$.*

Proof. Every element $p \in \mathbb{N}[X]$ has a unique representation as a finite sum of monomials:

$$p = \sum_{i=1}^{n} s_i m_i,$$

where $s_i \in \mathbb{N}$ and each m_i is a product of variables $x \in X$. Define $h : \mathbb{N}[X] \to S$ by replacing each x with $f(x)$ and performing the additions and multiplications in S:

$$h(p) = \sum_{i=1}^{n} \left(s_i \cdot_S \prod_{x \in m_i} f(x) \right),$$

where $s_i \cdot_S y$ denotes the s_i-fold sum $y +_S \cdots +_S y$ in S.

By construction, $h(x) = f(x)$ for all $x \in X$. Furthermore, $h(0) = 0_S$ and $h(1) = 1_S$. Since addition and multiplication in $\mathbb{N}[X]$ are defined by the semiring axioms and the distributive law, and h is defined by applying the corresponding operations in S, then h must preserve addition and multiplication:

$$h(p + q) = h(p) +_S h(q), \quad h(p \cdot q) = h(p) \cdot_S h(q).$$

For uniqueness, if $\hat{h} : \mathbb{N}[X] \to S$ is any semiring homomorphism with $\hat{h}(x) = f(x)$ for all $x \in X$, then by induction on the structure of polynomials, we can show that $\hat{h}(p) = h(p)$ for all $p \in \mathbb{N}[X]$. □

A powerful technique for constructing new algebraic structures from existing ones involves the use of congruence relations and quotient structures. By identifying distinct elements as equivalent, we can effectively force the resulting structure to satisfy additional algebraic laws.

Definition 2.2.13 (Congruence relation). Let S be a semiring. An equivalence relation $\sim$ on S is a *congruence relation* if, for all $s, s', t, t' \in S$:

$$s \sim s' \text{ and } t \sim t' \implies (s + t) \sim (s' + t') \text{ and } (s \cdot t) \sim (s' \cdot t').$$

▽

When an equivalence relation is a congruence, we can naturally define a semiring structure on the set of the equivalence classes.

Definition 2.2.14 (Quotient semiring). Given a semiring S and a congruence relation $\sim$, the *quotient semiring* $S/\sim$ is the set of equivalence classes $[s] = \{t \in S \mid s \sim t\}$, equipped with the operations:

$$[s] + [t] = [s + t] \quad \text{and} \quad [s] \cdot [t] = [s \cdot t].$$

These operations are well-defined because the congruence relation $\sim$ is compatible with the addition and multiplication of S. ▽

A standard application of this construction is to impose a specific property on an existing structure.

Example 2.2.15. The Boolean semiring $\mathbb{B}$ can be derived from the natural numbers $\mathbb{N}$ by taking the quotient with respect to the smallest congruence containing the idempotence identity: $x + x \sim x$. This collapses the counting behaviour of $\mathbb{N}$ into a binary structure representing existence. ▽

For the algebraic background used in this section, the reader is referred to two standard references. The general theory of algebraic structures is covered in detail in [Lan02], while [Gol99] offers a comprehensive treatment of semirings. These sources provide further details on the underlying concepts.

CHAPTER 3

Semirings for provenance

In this chapter, we present the semiring-based framework for provenance, which forms the core of this work. We first formalise the notion of annotated relations and present a generalisation of the positive relational algebra to this annotated setting.

We then establish two important properties: that the annotation structure that emerges from the requirement to preserve fundamental relational algebra identities is that of a commutative semiring; and that query evaluation commutes with semiring homomorphisms.

Finally, we focus on the polynomial semiring, showing how it serves as the provenance domain from which a wide range of semantics can be obtained.

1. Semiring-annotated relations

In databases, provenance is commonly modelled by annotating relations with additional information, typically at the tuple level. This idea is neither novel nor exclusive to provenance: as early as 1984, Imieliński and Lipski [IL84] generalised the relational algebra to handle incomplete information by allowing tuples to carry Boolean formulae as annotations. Other familiar semantics can be viewed in the same light: under standard set semantics (where duplicates are ignored), tuples are annotated with truth values indicating presence or absence; under bag semantics (where duplicates are allowed), tuples are annotated with natural numbers recording multiplicities; and in probabilistic databases, tuples carry annotations from the interval $[0, 1]$ representing confidence scores.

In Chapter 2, a tuple was defined as a function from a finite set of attribute names to the underlying domain, $t : U \to \mathbb{D}$, and a relation as a finite set of tuples.

We generalise this notion by allowing tuples to carry annotations. We accomplish this by associating each U-tuple with an element drawn from a set of annotations S.

To enable algebraic manipulation of these annotations, we assume that S is equipped with additional structure. As discussed in the introduction, data can be combined in two fundamental ways: alternatively and jointly. To capture these modes of combination, we assume the existence of two abstract operations $(+, \cdot)$ on S. We further assume that S contains two distinguished elements, 0 and 1, representing absence and presence, respectively. Collectively, these assumptions define the algebraic structure $\mathcal{S} = (S, +, \cdot, 0, 1)$.[a]

Definition 3.1.1 ($\mathcal{S}$-relation). Let $\mathcal{S}$ be the algebraic structure $(S, +, \cdot, 0, 1)$ and let $\mathbb{D}^U$ denote the set of all U-tuples. An *$\mathcal{S}$-relation* is a function $R : \mathbb{D}^U \to S$ with finite support. The support of R is $supp(R) := \{t \in \mathbb{D}^U \mid R(t) \neq 0\}$, i.e. the set of tuples that do not map to 0. An *$\mathcal{S}$-database* is a finite set of $\mathcal{S}$-relations. ▽

Building on these assumptions, Green et al. [Gre+07] generalised the operations of the positive relational algebra to relations annotated with elements of such an annotation set.

Definition 3.1.2 ($\mathcal{S}$-relational algebra). The positive relational algebra (SPJRU) operations with respect to $\mathcal{S}$-relations, which we call the *$\mathcal{S}$-relational algebra*, are defined as follows:

[a] Much of the early literature uses K instead of S.

- Union: If $R_i : \mathbb{D}^U \to S$ for $i \in \{1, 2\}$, then $R_1 \cup R_2 : \mathbb{D}^U \to S$ is defined as:
$$(R_1 \cup R_2)(t) = R_1(t) + R_2(t).$$
- Projection: If $R : \mathbb{D}^U \to S$ and $V \subseteq U$, then $\pi_V(R) : \mathbb{D}^V \to S$ is defined as:
$$(\pi_V(R))(t) = \sum_{t' \in supp(R) \text{ and } t = t'[V]} R(t'),$$
with $t'[V]$ denoting the restriction of the U-tuple t' to the attributes in V.
- Selection: If $R : \mathbb{D}^U \to S$ and θ maps each U-tuple to either 0 or 1, then $\sigma_\theta(R) : \mathbb{D}^U \to S$ is defined as:
$$(\sigma_\theta(R))(t) = R(t) \cdot \theta(t).$$
- Natural join: If $R_i : \mathbb{D}^{U_i} \to S$ for $i \in \{1, 2\}$, then $R_1 \bowtie R_2 : \mathbb{D}^{U_1 \cup U_2} \to S$ is defined as:
$$(R_1 \bowtie R_2)(t) = R_1(t[U_1]) \cdot R_2(t[U_2]).$$
- Renaming: If $R : \mathbb{D}^U \to S$ and β is a bijection $\beta : U \to U'$, then $\rho_\beta(R) : \mathbb{D}^{U'} \to S$ is defined as:
$$(\rho_\beta(R))(t) = R(t \circ \beta).$$

▽

PROPOSITION 3.1.3. *The operations of the $\mathcal{S}$-relational algebra preserve the finiteness of supports. Therefore they map $\mathcal{S}$-relations to $\mathcal{S}$-relations.*

The following example illustrates the $\mathcal{S}$-relational algebra in action.

EXAMPLE 3.1.4. Let S be an annotation domain satisfying the assumptions given above. Consider the relations *Orders* and *Customers*, represented in the tables below:

cust_id	item_id	price	
vsdjp	rdfhj-fgsdh	30	o_1
bjvks	qaerp-jckzn	40	o_2
hcbja	dvbkh-kjsna	20	o_3
hcbja	tyqui-vaslw	20	o_4

cust_id	name	
vsdjp	Aditya	c_1
bjvks	Luigi	c_2
hcbja	Luca	c_3
kocwb	Yiannis	c_4

FIGURE 2. Orders and Customers relations

Also consider the following query:

QUERY 3.1.5. Find all x such that x is the name of a customer with at least one order.

The query can be expressed in relational algebra as $\pi_{name}(Orders \bowtie Customers)$. The result of this query is the following relation, where the annotations from the original relations propagate through the operations:

name	
Aditya	$o_1 \cdot c_1$
Luigi	$o_2 \cdot c_2$
Luca	$o_3 \cdot c_3 + o_4 \cdot c_3$

▽

Many of the standard identities of relational algebra are preserved in the $\mathcal{S}$-relational setting precisely when $\mathcal{S}$ forms a commutative semiring. Thus, the axioms of commutative semirings are not imposed arbitrarily, but arise naturally from the requirement that fundamental relational algebra identities continue to hold in the annotated setting.

PROPOSITION 3.1.6. *The following identities hold for the $\mathcal{S}$-relational algebra if and only if $\mathcal{S}$ is a commutative semiring:*

- *Union is associative, commutative, and has identity $\mathbf{0}_U : \mathbb{D}^U \to S$ defined by $\mathbf{0}_U(t) = 0$.*
- *Join is associative, commutative, and has identity $\mathbf{1} : \mathbb{D}^\emptyset \to S$ defined by $\mathbf{1}(\langle\rangle) = 1$.*
- *Join distributes over union.*
- *$\mathbf{0}_U$ is a zero for join.*

PROOF. ($\Rightarrow$) We assume that the $\mathcal{S}$-relational algebra identities hold for all $\mathcal{S}$-relations and prove that $\mathcal{S}$ must satisfy the axioms of a commutative semiring. Let a, b, c be arbitrary elements of S and let t be a U-tuple. We construct the single-tuple $\mathcal{S}$-relations $R_a = \{(t, a)\}$, $R_b = \{(t, b)\}$, and $R_c = \{(t, c)\}$ over U. Since the relational algebra identities hold for all relations, they hold for R_a, R_b, and R_c. We evaluate both sides of these identities at the tuple t:

- If $R_a \cup R_b = R_b \cup R_a$, then $a + b = b + a$. Thus, $+$ is commutative.
- If $(R_a \cup R_b) \cup R_c = R_a \cup (R_b \cup R_c)$, then $(a + b) + c = a + (b + c)$. Thus, $+$ is associative.
- If $R_a \cup \mathbf{0}_U = R_a$, then $a + 0 = a$. Thus, 0 is the additive identity.
- If $R_a \bowtie R_b = R_b \bowtie R_a$, then $a \cdot b = b \cdot a$. Thus, $\cdot$ is commutative.
- If $(R_a \bowtie R_b) \bowtie R_c = R_a \bowtie (R_b \bowtie R_c)$, then $(a \cdot b) \cdot c = a \cdot (b \cdot c)$. Thus, $\cdot$ is associative.
- If $R_a \bowtie \mathbf{1} = R_a$, then $a \cdot 1 = a$. Thus, 1 is the multiplicative identity.
- If $R_a \bowtie (R_b \cup R_c) = (R_a \bowtie R_b) \cup (R_a \bowtie R_c)$, then $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$. Thus, $\cdot$ distributes over $+$.
- If $R_a \bowtie \mathbf{0}_U = \mathbf{0}_U$, then $a \cdot 0 = 0$. Thus, 0 is a zero for multiplication.

($\Leftarrow$) We assume that $\mathcal{S} = (S, +, \cdot, 0, 1)$ is a commutative semiring and prove that the identities of the $\mathcal{S}$-relational algebra hold by lifting the semiring axioms to the relation level. To show two relations are equal, we verify that their annotations match for every tuple t in their respective domain.

First, consider union. For any relations R, Q over U and any $t \in \mathbb{D}^U$, we have $(R \cup Q)(t) = R(t) + Q(t)$. The commutativity and associativity of $+$ in $\mathcal{S}$ imply that union is commutative and associative. Furthermore, $(R \cup \mathbf{0}_U)(t) = R(t) + \mathbf{0}_U(t) = R(t) + 0 = R(t)$, confirming $\mathbf{0}_U$ as the additive identity.

Next, consider natural join. For any relation R over U, any relation Q over V, and any $t \in \mathbb{D}^{U \cup V}$, we have $(R \bowtie Q)(t) = R(t[U]) \cdot Q(t[V])$. The commutativity and associativity of $\cdot$ in $\mathcal{S}$ imply that join is commutative and associative. To show $\mathbf{1}$ is the identity for join, we notice that for any $t \in \mathbb{D}^U$, the join definition yields $(R \bowtie \mathbf{1})(t) = R(t[U]) \cdot \mathbf{1}(t[\emptyset])$. Since $t[U] = t$ and the restriction $t[\emptyset]$ is the empty tuple $\langle\rangle$ (with $\mathbf{1}(\langle\rangle) = 1$), this simplifies to $R(t) \cdot 1 = R(t)$.

Finally, distributivity and the zero property. For any $t \in \mathbb{D}^{U \cup V}$, the annotation of $(R \bowtie (Q_1 \cup Q_2))(t)$ is $R(t[U]) \cdot (Q_1(t[V]) + Q_2(t[V]))$. By applying the distributive law of the semiring, this expression expands to $R(t[U]) \cdot Q_1(t[V]) + R(t[U]) \cdot Q_2(t[V])$, which is the definition of $(R \bowtie Q_1)(t) + (R \bowtie Q_2)(t)$. Lastly, $R \bowtie \mathbf{0}_U = \mathbf{0}_U$ holds because for any $t \in \mathbb{D}^U$, the join definition yields $(R \bowtie \mathbf{0}_U)(t) = R(t) \cdot \mathbf{0}_U(t) = R(t) \cdot 0$. This evaluates to 0 by the zero property of the semiring, matching $\mathbf{0}_U(t)$. □

A natural question arises regarding the choice of these specific identities, given that relational algebra admits a much larger set of algebraic equivalences. The justification for this is twofold [Ams+11a].

First, this particular subset of identities constitutes the core algebraic properties shared by both set and bag semantics. While certain identities, such as the idempotence of union, hold under set semantics but fail under bag semantics, the identities selected here represent the common denominator of both settings. Second, a majority of other valid equivalences in the positive relational algebra can be derived as direct consequences of these identities.

A central result is the fundamental theorem of the $\mathcal{S}$-relational algebra, which states that query evaluation commutes with semiring homomorphisms. Intuitively, it shows that the $\mathcal{S}$-relational algebra operations are independent of the underlying semiring. This result guarantees that we can first compute a query's provenance in a general semiring and then specialise it to any concrete semiring without changing the result, allowing us to reason consistently about provenance across different semirings.

REMARK. Any semiring homomorphism $h : \mathcal{S} \to \mathcal{S}'$ can be lifted to a map h_{Rel} on $\mathcal{S}$-relations, and by extension on $\mathcal{S}$-databases, by applying h to each tuple annotation.

THEOREM 3.1.7 (Fundamental theorem). *Let $\mathcal{S}, \mathcal{S}'$ be commutative semirings and let $h : S \to S'$ be a map. The identity $Q(h_{Rel}(D)) = h_{Rel}(Q(D))$ holds for every $\mathcal{S}$-relational algebra query Q and every $\mathcal{S}$-database D if and only if h is a semiring homomorphism.*

PROOF. ($\Rightarrow$) If h_{Rel} commutes with all $\mathcal{S}$-relational algebra queries, it must specifically commute with union, natural join, and their respective identities. By Proposition 3.1.6, these operations correspond to the semiring operations $(+, \cdot, 0, 1)$. Therefore, h preserves the additive and multiplicative structure of the semiring, making it a semiring homomorphism.

($\Leftarrow$) Assume that $h : \mathcal{S} \to \mathcal{S}'$ is a semiring homomorphism. We show that $Q(h_{Rel}(D)) = h_{Rel}(Q(D))$ for every query Q by induction on its structure. In the base case where Q is a single relation from the database, the identity holds by definition. For the inductive step, we assume that the statement holds for all sub-queries and perform case analysis on the last applied operation:

- *Union.* Let $Q = Q_1 \cup Q_2$. By the definition of union, the induction hypothesis, and the preservation of addition by h, for any tuple t:

$$\begin{aligned} Q(h_{Rel}(D))(t) &= Q_1(h_{Rel}(D))(t) + Q_2(h_{Rel}(D))(t) \\ &= h(Q_1(D)(t)) + h(Q_2(D)(t)) \\ &= h(Q_1(D)(t) + Q_2(D)(t)) \\ &= h_{Rel}(Q(D))(t). \end{aligned}$$

- *Projection.* Let $Q = \pi_V(Q')$. By the definition of projection, the induction hypothesis, and the fact that h preserves finite sums:

$$\begin{aligned} Q(h_{Rel}(D))(t) &= \sum_{t=t'[V]} Q'(h_{Rel}(D))(t') \\ &= \sum_{t=t'[V]} h(Q'(D)(t')) \\ &= h\left(\sum_{t=t'[V]} Q'(D)(t')\right) \\ &= h_{Rel}(Q(D))(t). \end{aligned}$$

- *Selection.* Let $Q = \sigma_\theta(Q')$. By the definition of selection and the induction hypothesis:

$$\begin{aligned} Q(h_{Rel}(D))(t) &= Q'(h_{Rel}(D))(t) \cdot \theta(t) \\ &= h(Q'(D)(t)) \cdot \theta(t). \end{aligned}$$

  Since $\theta(t) \in \{0, 1\}$ and h preserves the identities ($h(0) = 0$ and $h(1) = 1$), then $h(\theta(t)) = \theta(t)$. Using the preservation of multiplication:

$$\begin{aligned} h(Q'(D)(t)) \cdot \theta(t) &= h(Q'(D)(t)) \cdot h(\theta(t)) \\ &= h(Q'(D)(t) \cdot \theta(t)) \\ &= h_{Rel}(Q(D))(t). \end{aligned}$$

- *Natural join.* Let $Q = Q_1 \bowtie Q_2$. By the definition of natural join, the induction hypothesis, and the preservation of multiplication by h:

$$\begin{aligned} Q(h_{Rel}(D))(t) &= Q_1(h_{Rel}(D))(t_1) \cdot Q_2(h_{Rel}(D))(t_2) \\ &= h(Q_1(D)(t_1)) \cdot h(Q_2(D)(t_2)) \\ &= h(Q_1(D)(t_1) \cdot Q_2(D)(t_2)) \\ &= h_{Rel}(Q(D))(t). \end{aligned}$$

- *Renaming.* Let $Q = \rho_\beta(Q')$. As renaming does not modify annotations, it commutes with h_{Rel} as a direct consequence of the induction hypothesis:

$$Q(h_{Rel}(D)) = \rho_\beta(h_{Rel}(Q'(D))) = h_{Rel}(\rho_\beta(Q'(D))) = h_{Rel}(Q(D)).$$

□

The fundamental theorem can also be expressed diagrammatically [GT17]. Let h be a semiring homomorphism. Then, for any $\mathcal{S}$-relational algebra query Q, the diagram in Figure 3 commutes:

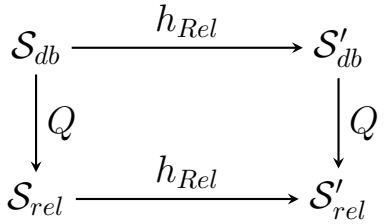


FIGURE 3. Fundamental theorem of the $\mathcal{S}$-relational algebra

2. One semiring to rule them all

A corollary of the fundamental theorem, together with Proposition 2.2.12, is the existence of a universal representation of provenance for $\mathcal{S}$-relational algebra queries.

More precisely, the polynomial semiring $\mathbb{N}[X]$ provides a general annotation domain in which the result of any $\mathcal{S}$-relational algebra query can be expressed symbolically. Evaluating a query in $\mathbb{N}[X]$ yields, for each output tuple, a polynomial whose variables correspond to input tuple annotations and whose algebraic structure records how the tuple was derived by the query.

By the universal property of the polynomial semiring, for every commutative semiring $\mathcal{S}$ and every assignment of variables $v : X \to S$, there exists a semiring homomorphism $h : \mathbb{N}[X] \to \mathcal{S}$ uniquely extending v. The fundamental theorem ensures that applying such a homomorphism to the $\mathbb{N}[X]$-valued query result yields exactly the same result as evaluating the query directly over the semiring $\mathcal{S}$. In this sense, query evaluation over an arbitrary semiring $\mathcal{S}$ factors through evaluation over $\mathbb{N}[X]$.

This separation between provenance capture and provenance interpretation allows reuse of the same symbolic representation to support a wide range of annotation semantics. For this reason, we refer to the polynomial semiring $\mathbb{N}[X]$ as the *provenance semiring* for the $\mathcal{S}$-relational algebra.

COROLLARY 3.2.1. *Let D be an $\mathcal{S}$-database and let $X = \{x_t \mid R \in D, t \in supp(R)\}$ be a set of distinct tuple identifiers. Let $\bar{D}$ be the abstractly tagged version of D, where each tuple t in the support of each relation is annotated with its corresponding variable $x_t \in X$.*

If $v : X \to S$ is the valuation defined by $v(x_t) = R(t)$, then for any query Q:

$$Q(D) = h_{Rel}\big(Q(\bar{D})\big),$$

where $h : \mathbb{N}[X] \to \mathcal{S}$ is the unique semiring homomorphism extending v.

PROOF. By construction, $\bar{D}$ is an $\mathbb{N}[X]$-database. Since $\mathbb{N}[X]$ is the free commutative semiring on X, there exists a unique semiring homomorphism $h : \mathbb{N}[X] \to \mathcal{S}$ such that $h(x_t) = v(x_t) = R(t)$. This means that $h_{Rel}(\bar{D}) = D$. By the fundamental theorem, query evaluation commutes with semiring homomorphisms. Therefore:

$$Q(D) = Q\big(h_{Rel}(\bar{D})\big) = h_{Rel}\big(Q(\bar{D})\big).$$

□

Indeed, the semirings we have encountered thus far can all be viewed as instances of the provenance semiring, realised by replacing the abstract annotations with elements from the specific semirings.

In each of these cases, the operations of the $\mathcal{S}$-relational algebra remain unchanged; what varies is the interpretation of the annotations via the chosen semiring.

EXAMPLE 3.2.2. Some interesting semirings include [GT17]:

- The Boolean semiring $\mathbb{B} = (\{0, 1\}, \vee, \wedge, 0, 1)$: Recovers standard set semantics. The elements 0 and 1 represent absence and presence, while addition ($\vee$) and multiplication ($\wedge$) represent disjunction and conjunction.
- The natural numbers semiring $\mathbb{N} = (\mathbb{N}, +, \cdot, 0, 1)$: Models bag semantics. Tuple annotations represent multiplicities, with union summing ($+$) and join multiplying ($\cdot$) these values.
- The tropical semiring $\mathbb{T} = (\mathbb{R}_+ \cup \{\infty\}, \min, +, \infty, 0)$: Used for cost and shortest-path calculations. Addition (min) selects the cheapest alternative path, while multiplication ($+$) accumulates costs along a path.
- The Viterbi semiring $\mathbb{V} = ([0, 1], \max, \cdot, 0, 1)$: Models confidence scores and probabilities. Addition (max) finds the most likely outcome, while multiplication ($\cdot$) combines independent probabilities. It is isomorphic to $\mathbb{T}$ via the mapping $x \mapsto -\ln x$. In practice, it is used in hidden Markov models [Rab89].
- The access control semiring $\mathbb{A} = (\{1 < C < S < T < 0\}, \min, \max, 0, 1)$: Models security clearance levels, ranging from public (1) to private (0). Addition (min) returns the least restrictive clearance needed across alternative paths, while multiplication (max) computes the highest clearance required to join data.
- The fuzzy semiring $\mathbb{F} = ([0, 1], \max, \min, 0, 1)$: Used for uncertainty propagation and fuzzy queries. Addition (max) selects the most confident value among alternatives, while multiplication (min) represents the conjunction of evidence when combining joint constraints.

▽

Beyond the concrete semirings discussed above, the provenance semiring provides a unifying framework for capturing multiple provenance models, generalising each of them. These models form a hierarchy that ranges from the most informative to the least. We illustrate with a diagram which summarises results from Green's work in [Gre11].

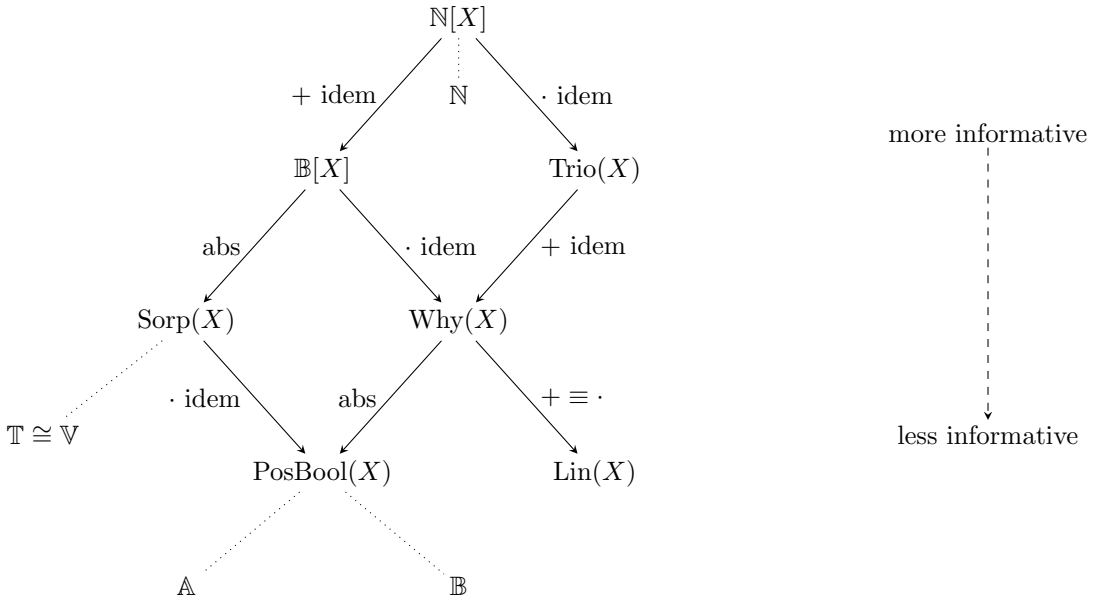


FIGURE 4. Semiring provenance hierarchy [GT17]

In Figure 4, solid arrows denote the existence of a surjective semiring homomorphism, while the dotted arrows connect concrete semirings to the least informative provenance models that can still support their applications.

Coupled with Theorem 3.1.7, this hierarchy gives a clear picture of the relative informativeness of the various provenance models, since provenance computations for models lower in the hierarchy can always be factored through computations involving models above them in the hierarchy. While provenance models lower in the hierarchy are less informative, they may be preferred when more efficient computation is desired.

More specifically, these provenance models are:

- The Boolean provenance semiring, $\mathbb{B}[X]$, consists of polynomials analogous to those in $\mathbb{N}[X]$, but with Boolean coefficients, i.e. with idempotent addition.
- Trio, a system developed at Stanford for managing provenance in uncertain databases [Ben+08]. The Trio provenance model can be obtained from $\mathbb{N}[X]$ by making multiplication idempotent, instead of addition.
- If both operations are made idempotent, we can retrieve the why-provenance model developed by Buneman et al. [Bun+01]. More specifically, we can use the following semiring to obtain why-provenance [Che+09]: $\text{Why}(X) = (\mathcal{P}(\mathcal{P}(X)), \cup, \Cup, \emptyset, \{\emptyset\})$, where $\Cup$ takes all the pairwise unions of two collections: $\{s \cup t \mid s \in S, t \in T\}$.
- From $\mathbb{B}[X]$ we can obtain Sorp(X) by adopting the absorption law $a + a \cdot b = a$. Sorp(X) consists of absorptive polynomials and is the free absorptive commutative semiring over X.
- Further down is PosBool(X), consisting of Boolean expressions constructed using only conjunction and disjunction.
- Finally, we can obtain the lineage provenance model Lin(X), as described in [Cui+00], by making addition and multiplication equal in the Why(X) provenance model. The corresponding semiring is $\text{Lin}(X) = (\mathcal{P}(X) \cup \{\bot\}, \cup_L, \cup_S, \bot, \emptyset)$,

where $\cup_L$ denotes lazy union and $\cup_S$ denotes strict union, as defined by Cheney, Chiticariu, and Tan [Che+09].

EXAMPLE 3.2.3. As a further illustration of the informativeness of the different provenance models, consider the example $\mathcal{S}$-relation R in Figure 5a.[b]

Furthermore, consider the following Datalog program:

QUERY 3.2.4.

$$Q(x, z) \leftarrow R(x, y, u) \wedge R(v, y, z)$$
$$Q(x, z) \leftarrow R(x, u, z) \wedge R(v, y, z)$$

The different provenance annotations for this query on R are shown in Figure 5b [Gre11].

a	b	c	**p**
d	b	e	**r**
f	g	e	**s**

(A) Example $\mathcal{S}$-relation R

		$\mathbb{N}[X]$	$\mathbb{B}[X]$	Trio(X)	Why(X)	PosBool(X)	Lin(X)
a	c	$2p^2$	p^2	$2p$	$\{\{p\}\}$	p	$\{p\}$
a	e	pr	pr	pr	$\{\{p, r\}\}$	$p \wedge r$	$\{p, r\}$
d	c	pr	pr	pr	$\{\{p, r\}\}$	$p \wedge r$	$\{p, r\}$
d	e	$2r^2 + rs$	$r^2 + rs$	$2r + rs$	$\{\{r\}, \{r, s\}\}$	r	$\{r, s\}$
f	e	$2s^2 + rs$	$s^2 + rs$	$2s + rs$	$\{\{s\}, \{r, s\}\}$	s	$\{r, s\}$

(B) Provenance annotations for Query 3.2.4

FIGURE 5. Query evaluation under different provenance models

▽

For a more comprehensive survey of the broader provenance landscape, including techniques for capturing, storing, and querying provenance; connections to other fields; and applications, the reader is referred to [Gla21].

While the $\mathcal{S}$-relational algebra provides a robust framework for core operations, it represents only the starting point for provenance-aware data processing. To meet the demands of modern workloads, this framework must be extended to handle more expressive queries. In the following chapters, we will investigate extensions of the $\mathcal{S}$-relational algebra, specifically addressing the challenges of aggregation and difference.

[b] Rather than using contrived names for the attributes, we omit them altogether.

CHAPTER 4

Aggregation

Aggregation is a key operation in data analysis, as it enables the computation of summary information over collections of facts. Rather than inspecting individual tuples in isolation, aggregation combines values across sets of tuples into a single result.

While aggregation was present in SQL by the mid-1970s, it was formalised as an extension of the relational algebra and calculus in 1982 by Anthony Klug [Klu82]. Klug introduced a formal operator for grouping and aggregation, which we denote by $\gamma_{L,f(A)}(R)$.

Let R be a relation over attribute set U. Given a list of grouping attributes $L \subseteq U$, the aggregation operator collects together all tuples that share the same values for those attributes. If no grouping attributes are specified, the entire relation R is treated as a single group. For each group, the operator then applies an aggregate function f (e.g. SUM, MIN, or AVG) to a target attribute $A \in U$. The result is a new relation containing one tuple per group, together with the corresponding aggregate value.

For aggregation to make sense, the standard set semantics, where duplicate elements are ignored, is insufficient. To calculate a sum, for instance, the number of times a value appears is crucial. This necessitates moving from set semantics to bag semantics.

Under bag semantics, relations are treated as multisets rather than simple sets. In this framework, each tuple may appear multiple times, rather than simply being present or absent.

The following example illustrates this point.

Example 4.0.1. Consider a relation R representing university employees and their salaries:

emp_id	dept	salary
52435	History	50
57252	History	70
80959	Maths	60
82534	Maths	60

Using the aggregation operator, we can compute the total salary per department as $\gamma_{dept,\ \mathrm{SUM}(salary)}(R)$. The result is the following relation:

dept	SUM(salary)
History	120
Maths	120

Note that the two identical salary values in the Maths department both contribute to the sum. Under set semantics, where duplicates are ignored, this result could not be obtained. ▽

When extending the semiring framework to support aggregation, it is necessary to articulate the desiderata that such an extension should satisfy. As a baseline, we require that the extension remain compatible with both set and bag semantics and that the fundamental theorem continues to hold.

As we shall see, the semiring structure on tuple annotations alone is insufficient to meet these requirements. Aggregation combines data values with annotations, and this interaction cannot be captured within the semiring framework without assuming additional structure.

To address this, we require a suitable notion of how annotations act on values. Amsterdamer, Deutch, and Tannen [Ams+11b] propose an approach that satisfies these desiderata by allowing annotations to act on the data values being aggregated, rather than merely annotating the tuples themselves. This leads to a semimodule structure where the annotation semiring acts on the domain of aggregate values, providing a uniform framework for aggregating annotated data. This framework will be the main focus of this chapter.

This representation is motivated by considering how aggregation behaves under bag semantics. Let us revisit the *Orders* relation from Figure 2, which has been reproduced below. Suppose we perform a SUM-aggregation on the price attribute, grouping by cust_id. For the group with cust_id `hcbja`, the aggregated value is computed as $o_3 \cdot 20 + o_4 \cdot 20$, where o_3 and o_4 are the multiplicities of the corresponding tuples, scaling the data values.

cust_id	item_id	price	
vsdjp	rdfhj-fgsdh	30	o_1
bjvks	qaerp-jckzn	40	o_2
hcbja	dvbkh-kjsna	20	o_3
hcbja	tyqui-vaslw	20	o_4

If, for example, $o_3 = 1$ and $o_4 = 2$, then the aggregate evaluates to $1 \cdot 20 + 2 \cdot 20 = 60$, as expected under bag semantics. However, if the input relation is annotated with polynomials rather than multiplicities, the aggregated result is no longer a concrete number, but an expression whose value depends on the interpretation of the provenance annotations.

Amsterdamer et al. address this challenge by introducing symbolic expressions that record how the result of an aggregation function is constructed. Rather than directly multiplying a tuple annotation with an attribute value, they form formal expressions by pairing each value with the annotation of the tuple to which it belongs. For instance, $o_3 \cdot 20$ is replaced by the formal expression $o_3 \otimes 20$, and the aggregate value becomes $o_3 \otimes 20 + o_4 \otimes 20$, where $\otimes$ denotes a formal operation that combines annotations with data values.

These expressions live in a tensor product domain where the data values are embedded into a semimodule over the annotation semiring. The choice of a tensor product construction is motivated by a universal property (which we formally enunciate in the next section): it represents the most general semimodule that can be constructed from the semiring S and the domain of aggregate values.

Before introducing this construction in detail, however, we first demonstrate that extending the semiring framework with aggregation breaks the fundamental theorem. This motivates the need for the additional algebraic structure provided by semimodules and the tensor product construction.

Proposition 4.0.2. *There exists no $\mathcal{S}$-relation semantics for MAX-(or MIN-)aggregation that is both set-compatible and commutes with semiring homomorphisms. Likewise, there exists no $\mathcal{S}$-relation semantics for SUM-aggregation that is both bag-compatible and commutes with semiring homomorphisms.*

Proof. We first consider MAX-aggregation. Assume for contradiction that there exists a semantics Q for MAX-aggregation that is both set-compatible and commutes with semiring homomorphisms.

Let D be an $\mathbb{N}[a, b]$-database containing a single-attribute $\mathbb{N}[a, b]$-relation R. Let $t_1 = (5)$ and $t_2 = (6)$ be the tuples in the support of R, with $R(t_1) = a$ and $R(t_2) = b$.

Now let $h : \mathbb{N}[a, b] \to \mathbb{B}$ be the homomorphism defined by $h(a) = 1$ and $h(b) = 0$.

Evaluating Q first yields t_2 as the maximum with annotation b. After that, applying h_{Rel} gives us an annotation of 0 for t_2. Conversely, applying h_{Rel} first gives $\{(t_1, 1), (t_2, 0)\}$. Therefore we are left with only t_1 in the support. Evaluating Q yields t_1 as the maximum with annotation 1.

Therefore, $h_{Rel}(Q(D)) \neq Q(h_{Rel}(D))$, contradicting the assumption that query evaluation commutes with semiring homomorphisms. The argument for MIN-aggregation is analogous.

We now consider SUM-aggregation. Assume for contradiction that there exists a semantics Q for SUM-aggregation that is both bag-compatible and commutes with semiring homomorphisms.

Let D be an $\mathbb{N}[a,b]$-database containing a single-attribute $\mathbb{N}[a,b]$-relation R. Let $t_1 = (2)$ and $t_2 = (3)$ be the tuples in the support of R, and let $R(t_1) = a$ and $R(t_2) = b$.

Since Q returns an $\mathcal{S}$-relation, its output must consist of data values in $\mathbb{D}$. Hence, the query result must choose some concrete value in the data domain independent of the annotation domain.

Now let $h : \mathbb{N}[a,b] \to \mathbb{N}$ be the homomorphism defined by $h(a) = 1$ and $h(b) = 0$.

Applying h_{Rel} first gives $\{(t_1, 1), (t_2, 0)\}$. Evaluating Q yields a result tuple with value 2 and annotation 1. Conversely, evaluating Q and then applying h_{Rel} yields a result tuple with value 5 and annotation 1.

Therefore, $Q(h_{Rel}(D)) \neq h_{Rel}(Q(D))$, contradicting the assumption. □

1. Semimodules

To address these limitations, we shift to a framework where data values and annotations are no longer treated as separate entities. By embedding the value domain into a semimodule over the annotation semiring, we can represent results as symbolic expressions such as $o_3 \otimes 20 + o_4 \otimes 20$.

We notice that such expressions rely on two types of algebraic operations [Gla21]:

- Aggregation: an operation used to combine data values (e.g. addition).
- Pairing: a map that scales a data value by a semiring annotation.

More formally, we assume that the aggregation values form a commutative monoid. For instance,

$$\texttt{SUM} = (\mathbb{R}, +, 0), \qquad \texttt{MIN} = (\mathbb{R} \cup \{\infty\}, \min, \infty), \qquad \texttt{MAX} = (\mathbb{R} \cup \{-\infty\}, \max, -\infty).$$

We also require the pairing map $* : S \times M \to M$, for a commutative monoid M and a commutative semiring S, to satisfy certain algebraic axioms. Conveniently, these are the axioms of a semimodule.

Definition 4.1.1 (Semimodule). Let S be a commutative semiring. An *S-semimodule* $(M, +_M, 0_M, *)$ is a commutative monoid $(M, +_M, 0_M)$ equipped with a map $* : S \times M \to M$, called scalar multiplication, such that for all $r, s \in S$ and $m, n \in M$:

$$\begin{aligned} r * (m +_M n) &= r * m +_M r * n \\ (r +_S s) * m &= r * m +_M s * m \\ (r \cdot_S s) * m &= r * (s * m) \\ 1_S * m &= m \\ 0_S * m = 0_M \quad &\text{and} \quad r * 0_M = 0_M \end{aligned}$$

▽

Remark. Note that if S is a field (a ring where the nonzero elements form an abelian group under multiplication), an S-semimodule is simply a vector space. If S is a commutative ring, it is a module. Thus, semimodules provide a generalisation for the case where the scalars form a semiring.

These axioms ensure that aggregation interacts coherently with both the semiring structure of the annotations and the monoid structure of the aggregation domain. In particular, they generalise the behaviour of multiplication under bag semantics.

We say that a commutative monoid $(M, +_M, 0_M)$ can be *extended* to an S-semimodule if there exists a map $* : S \times M \to M$ such that $(M, +_M, 0_M, *)$ satisfies the semimodule axioms. We now record two useful facts about semimodules.

PROPOSITION 4.1.2. *Every commutative monoid can be extended to an $\mathbb{N}$-semimodule.*

PROOF. Let $(M, +_M, 0_M)$ be a commutative monoid. For any $n \in \mathbb{N}$ and $m \in M$, we define the scalar multiplication $* : \mathbb{N} \times M \to M$ as the n-fold sum of m:

$$0 * m = 0_M$$
$$(n+1) * m = (n * m) +_M m$$

By induction on n, it is straightforward to verify that this definition of $*$ satisfies the semimodule axioms. □

PROPOSITION 4.1.3. *A commutative monoid M can be extended to a $\mathbb{B}$-semimodule if and only if M is idempotent.*

PROOF. ($\Rightarrow$) Suppose $(M, +_M, 0_M, *)$ is a $\mathbb{B}$-semimodule. By the semimodule axioms, the identity $(s + s) * m = (s * m) +_M (s * m)$ must hold for any $s \in \mathbb{B}$. In the Boolean semiring, $1 + 1 = 1$. Substituting $s = 1$ into the axiom yields $1 * m = (1 * m) +_M (1 * m)$. Since $1 * m = m$ by the identity law, this simplifies to $m = m +_M m$, showing that M is idempotent.

($\Leftarrow$) Suppose M is an idempotent commutative monoid. We define the scalar multiplication map $* : \mathbb{B} \times M \to M$ by $1 * m = m$ and $0 * m = 0_M$. The semimodule axioms follow immediately from the properties of the Boolean semiring together with the idempotence of the monoid operation. □

Since, by Proposition 4.1.2, every commutative monoid can be extended to an $\mathbb{N}$-semimodule, the framework recovers the usual bag semantics for aggregation: in particular, it supports standard operations such as `SUM`, `MIN`, and `MAX` over bags. Moreover, since `MIN` and `MAX` form idempotent commutative monoids, using Proposition 4.1.3, they can be extended to $\mathbb{B}$-semimodules and therefore yield the standard set semantics of `MIN` and `MAX` over sets.

In general however, an arbitrary commutative monoid M does not have an inherent S-semimodule structure. To make it such we define an extended aggregation domain.

The process is as follows: We consider the free commutative monoid generated by $S \times M$. An element of this monoid is a formal sum of the form:

$$\sum_{i=1}^{n} s_i \otimes m_i$$

where $s_i \in S$ and $m_i \in M$. We use the notation $s \otimes m$ instead of the pair (s, m) to emphasise the intended tensor-like relationship.

Next, we need to force these expressions to respect the underlying properties of both the monoid and the semiring. We define $S \otimes M$ as the quotient of this free monoid by the smallest congruence relation $\sim$ that satisfies the following semimodule laws:

$$s \otimes (m +_M n) \sim (s \otimes m) + (s \otimes n)$$
$$(s +_S r) \otimes m \sim (s \otimes m) + (r \otimes m)$$
$$s \otimes 0_M \sim 0_S \otimes m \sim 0$$

Finally, with the quotient space established, we can now equip these expressions with scalar multiplication $*$. It is defined by scaling the semiring component of each symbolic pair. For any $s \in S$:

$$s * \left(\sum_i s_i \otimes m_i \right) = \sum_i (s \cdot_S s_i) \otimes m_i.$$

The resulting algebraic structure $(S \otimes M, +, 0, *)$ is the *tensor product* of the semiring S and the monoid M. The original monoid M is mapped into this symbolic domain through $\iota : M \to S \otimes M$ defined as: $\iota(m) = 1_S \otimes m$.

Intuitively, the pairing $S \times M$ creates a shared domain where annotations and data values can coexist without yet interacting. The congruence relation synchronises their independent operations, and the scalar multiplication establishes how annotations scale the data. By enforcing these relations, we ensure that the resulting structure respects the properties of the underlying structures.

Beyond these structural rules, the tensor product $S \otimes M$ is characterised by a universal property: it is the most general S-semimodule constructed from M. More specifically, for any S-semimodule N and monoid homomorphism $f : M \to N$, there exists a unique S-semimodule homomorphism $\tilde{f} : S \otimes M \to N$ such that $\tilde{f}(\iota(m)) = f(m)$ for all $m \in M$. This property formalises the role of $S \otimes M$ as the provenance domain for aggregation.

We now extend the $\mathcal{S}$-relational algebra to an $(\mathcal{M}, \mathcal{S})$-relational algebra by first assuming that $S \otimes M \subseteq \mathbb{D}$. In this framework, the standard SPJRU operations remain identical to their definitions in the original $\mathcal{S}$-relational algebra. We only introduce operations for aggregation and grouping.

Definition 4.1.4 (Aggregation). The aggregation operation, AGG_M, acts on a single-attribute relation R where values are drawn from M. Let $supp(R) = \{m_1, \ldots, m_n\}$ be the set of input values with corresponding annotations $s_i = R(m_i)$. The result $AGG_M(R)$ is a single-tuple relation with annotation 1. The value v of this resulting tuple is defined using the semimodule structure:

$$v = \sum_{i=1}^{n} s_i * \iota(m_i) = \sum_{i=1}^{n} s_i \otimes m_i.$$

▽

This definition ensures that the aggregate result is a formal sum in $S \otimes M$ that preserves the provenance of each contributing tuple. This operation serves as the foundation for the more general grouping operation, which partitions a relation before applying aggregation to each group.

To define the grouping operation, we must determine the annotation of each output group. Under bag semantics, a group exists in the query output if and only if there is at least one contributing input tuple. The annotation of a group is computed by summing the annotations of its matching tuples. If this sum is nonzero (meaning the group contains at least one tuple from its support), the output annotation should collapse to 1, and be 0 otherwise. We thus introduce a unary operation on the semiring that acts as an indicator of this existence.

Definition 4.1.5 (δ-semiring). A *δ-semiring* is a commutative semiring $(S, +_S, \cdot_S, 0_S, 1_S)$ equipped with a unary operation $\delta_S : S \to S$ satisfying the following laws:

$$\delta_S(0_S) = 0_S$$
$$\delta_S(n \cdot 1_S) = 1_S \text{ for all } n \geq 1.$$

If S and S' are δ-semirings, a homomorphism between them is a semiring homomorphism $h : S \to S'$ such that $h(\delta_S(s)) = \delta_{S'}(h(s))$. ▽

These laws were chosen because they represent the minimal conditions for compatibility with both set and bag semantics, and for the fundamental theorem to continue to hold.

We can now give the definition of the grouping operation.

Definition 4.1.6 (Grouping). Let R be an S-relation over attribute set U. Let $U' \subseteq U$ be the set of grouping attributes and $U'' \subseteq U$ be the set of attributes to be aggregated, such that $U' \cap U'' = \emptyset$. For an output tuple t over $U' \cup U''$, let T be the set of input tuples that match t on the grouping attributes:

$$T = \{t' \in supp(R) \mid \forall u \in U'\ t'(u) = t(u)\}.$$

The grouping operation, $GB_{U',U''}(R) = R'$, is defined as follows:

$$R'(t) = \begin{cases} \delta_S\left(\sum_{t' \in T} R(t')\right) & T \neq \emptyset \text{ and } \forall u \in U''\ t(u) = \sum_{t' \in T} R(t') \otimes t'(u) \\ 0 & \text{otherwise.} \end{cases}$$

▽

Intuitively, for each attribute u in the aggregation attribute set U'', the value in the output tuple $t(u)$ is the formal sum $\sum R(t') \otimes t'(u)$. For the annotation, we are passing the combined provenance of all matching input tuples through δ.

We now turn to the fundamental theorem for the extended algebra, followed by two examples. To do so, we must first formalise how δ-semiring homomorphisms are lifted to act on the enriched structures.

Remark. Any δ-semiring homomorphism $h : S \to S'$ can be lifted to a homomorphism $h^M : S \otimes M \to S' \otimes M$ by applying h to the semiring part of the pairs in an $S \otimes M$-element:

$$h^M\left(\sum_i s_i \otimes m_i\right) = \sum_i h(s_i) \otimes m_i.$$

In turn, we can lift h to a map h_{Rel} on $(\mathcal{M}, \mathcal{S})$-relations. For a given relation R, $h_{Rel}(R)$ is obtained by replacing every tuple annotation $s \in S$ with $h(s)$, every aggregate data value belonging to the enriched domain $s \otimes m \in S \otimes M$ with $h^M(s \otimes m)$, while leaving all other values in R untouched.

Theorem 4.1.7 (Fundamental theorem). *Let S, S' be commutative δ-semirings and let $h : S \to S'$ be a map. The identity $Q(h_{Rel}(D)) = h_{Rel}(Q(D))$ holds for every $(\mathcal{M}, \mathcal{S})$-relational algebra query Q and every $(\mathcal{M}, \mathcal{S})$-database D if and only if h is a δ-semiring homomorphism.*

Proof. We prove the interesting direction. Assume h is a δ-semiring homomorphism. The proof proceeds by induction on the structure of the query Q. As the standard SPJRU operations are known to commute with semiring homomorphisms, we need only verify the cases for aggregation and grouping.

For the aggregation operation AGG_M, let R be an input relation with annotations s_i and values m_i. The result $Q(D)$ is a single-tuple relation with value $v = \sum s_i \otimes m_i$ and annotation 1_S. Applying h_{Rel} to this result transforms the annotation to $h(1_S) = 1_{S'}$ and the value to $h^M(v) = \sum h(s_i) \otimes m_i$. Conversely, applying h_{Rel} to the input database first, replaces each s_i with $h(s_i)$. Aggregating this transformed database yields the same value $\sum h(s_i) \otimes m_i$ with annotation $1_{S'}$. Thus, aggregation commutes with h.

Finally, we consider the grouping operation $GB_{U',U''}$. For an output tuple t, let T be the matching set of input tuples. The annotation of t in $h_{Rel}(Q(D))$ is $h(\delta_S(\sum_{t' \in T} R(t')))$. Since h is a δ-semiring homomorphism, this is equivalent to $\delta_{S'}(\sum h(R(t')))$, which is exactly the annotation computed by $Q(h_{Rel}(D))$. For the aggregated attributes $u \in U''$, the output values are formal sums of the form $\sum_{t' \in T} R(t') \otimes t'(u)$. Applying h_{Rel} involves the homomorphism h^M, which transforms $R(t')$ to $h(R(t'))$. This results in the formal

sum $\sum_{t' \in T} h(R(t')) \otimes t'(u)$, which is identical to the aggregate computed by Q when the input annotations have already been transformed by h. Therefore, grouping commutes with h. □

Example 4.1.8. Consider the *Orders* relation from Figure 2. Performing an aggregation over the price attribute results in a single-tuple relation with the following symbolic value in $S \otimes M$:

$$v = o_1 \otimes 30 + o_2 \otimes 40 + o_3 \otimes 20 + o_4 \otimes 20$$

The commutation with homomorphisms allows us to specialise this result for any specific semiring mapping. For instance, if $M = \texttt{SUM}$ and we apply a homomorphism h where $o_1 = 1$, $o_2 = 0$, and $o_3 = o_4 = 1$, the expression evaluates as follows:

$$\begin{aligned} h^M(v) &= 1 \otimes 30 + 0 \otimes 40 + 1 \otimes 20 + 1 \otimes 20 \\ &= 1 \otimes 30 + 1 \otimes (20 + 20) \\ &= 1 \otimes 70 \\ &= 70 \end{aligned}$$

By contrast, if we apply the same mapping but let $M = \texttt{MAX}$, the result simplifies to $1 \otimes \max(30, 20, 20) = 30$. ▽

Example 4.1.9. We now consider a grouping query $GB_{\{cust\_id\},\{price\}}$ on the *Orders* relation, using $M = \texttt{SUM}$. This operation partitions the relation by cust_id and aggregates their respective spending. The resulting $(\mathcal{M}, \mathcal{S})$-relation is shown below:

cust_id	price	
vsdjp	$o_1 \otimes 30$	$\delta(o_1)$
bjvks	$o_2 \otimes 40$	$\delta(o_2)$
hcbja	$o_3 \otimes 20 + o_4 \otimes 20$	$\delta(o_3 + o_4)$

The aggregated value for each customer is computed as a formal sum in the tensor domain. The annotations of the first two tuples evaluate to 1 provided the respective input rows exist. In the case of `hcbja`, we expect the group to exist if at least one of the contributing tuples is present. ▽

Although the tensor product construction facilitates the integration of annotations and data values, its utility depends on the faithfulness of the mapping $\iota : M \to S \otimes M$. While this mapping is injective for both set semantics (min/max) and bag semantics (sum), in many other cases it is not, meaning that distinct elements of the original monoid may become indistinguishable within the tensor domain.

Example 4.1.10. The mapping $\iota : \texttt{SUM} \to \mathbb{B} \otimes \texttt{SUM}$ is not injective: $\iota(4) = \iota(2 + 2) = \iota(2) + \iota(2) = (1_{\mathbb{B}} \otimes 2) + (1_{\mathbb{B}} \otimes 2) = (1_{\mathbb{B}} \vee 1_{\mathbb{B}}) \otimes 2 = 1_{\mathbb{B}} \otimes 2 = \iota(2)$ ▽

We now formalise this requirement and present two theorems establishing the conditions under which this property holds.

Definition 4.1.11 (Compatible). A commutative semiring S and a commutative monoid M are said to be *compatible* if $\iota : M \to S \otimes M$ is injective. ▽

The importance of the compatibility definition is that it ensures our algebraic construction is safe: when a result in the tensor domain $S \otimes M$ falls within the image of ι, we can uniquely and correctly interpret it as a standard data value from M.

Our first theorem establishes that idempotent monoids (such as `MIN` and `MAX`) are compatible with most common semirings. We define a *positive* semiring as one where for all $s, s' \in S$: if $s + s' = 0$ then $s = s' = 0$. Note that $\mathbb{B}, \mathbb{N}$, and $\mathbb{N}[X]$ are all positive.

Theorem 4.1.12. *If $(M, +_M, 0_M)$ is an idempotent commutative monoid, then M is compatible with any positive commutative semiring S.*

PROOF. To prove that ι is injective, it suffices to construct a (left-inverse) homomorphism $h : S \otimes M \to M$ such that $h \circ \iota$ is the identity function. Since S is positive, the map $f : S \to \mathbb{B}$ defined by $f(s) = 1$ if $s \neq 0$ and $f(0) = 0$ preserves addition and zero. Furthermore, by Proposition 4.1.3, M can be extended to a $\mathbb{B}$-semimodule with $*_{\mathbb{B}}$.

We define h by $h(s \otimes m) = f(s) *_{\mathbb{B}} m$. For any $m \in M$, we have:

$$(h \circ \iota)(m) = h(1_S \otimes m) = f(1_S) *_{\mathbb{B}} m = 1_{\mathbb{B}} *_{\mathbb{B}} m = m.$$

Since $h \circ \iota$ is the identity on M, ι is injective, and thus S and M are compatible. □

For non-idempotent monoids like `SUM`, compatibility is more restrictive. However, a sufficient condition exists for semirings that can be mapped to the natural numbers.

THEOREM 4.1.13. *Let S be a commutative semiring. If there exists a semiring homomorphism $f : S \to \mathbb{N}$, then S is compatible with any commutative monoid M.*

PROOF. By Proposition 4.1.2, every commutative monoid M can be extended to an $\mathbb{N}$-semimodule, with $*_{\mathbb{N}}$. We define a mapping $h : S \otimes M \to M$ by $h(s \otimes m) = f(s) *_{\mathbb{N}} m$. Evaluating this yields:

$$(h \circ \iota)(m) = h(1_S \otimes m) = f(1_S) *_{\mathbb{N}} m = 1_{\mathbb{N}} *_{\mathbb{N}} m = m.$$

As $h \circ \iota$ is the identity, ι is injective for any commutative monoid M. □

As we have seen, idempotent semirings are suitable for operations like `MIN` and `MAX`, but they are inherently incompatible with non-idempotent aggregations such as `SUM`. To accommodate such aggregates, we can construct a bag-extended semiring that preserves the multiplicative structure of S while allowing the additive structure to track multiplicities through the use of polynomials.

We begin with the semiring of polynomials $\mathbb{N}[S]$, where the variables are elements of S and the coefficients are natural numbers. While $\mathbb{N}[S]$ is compatible with all commutative monoids, it does not always respect the multiplicative structure and identity elements of the original semiring S. We define $S\mathbb{N}$ as the quotient of $\mathbb{N}[S]$ by the smallest congruence relation $\sim$ that satisfies the following identities for all $n \in \mathbb{N}$ and $s, s' \in S$:

$$(1 \cdot s) \cdot (1 \cdot s') \sim 1 \cdot (s \cdot_S s')$$
$$n \cdot 0_S \sim 0 \cdot s \sim 0$$
$$n \cdot 1_S \sim n$$

The existence of a homomorphism $f : S\mathbb{N} \to \mathbb{N}$ ensures that $S\mathbb{N}$ satisfies the condition for universal compatibility established in Theorem 4.1.13.

EXAMPLE 4.1.14. To illustrate this construction, consider an idempotent semiring S and the monoid $M = \texttt{SUM}$. Suppose a query identifies the value 10 through two distinct paths, annotated with a and b. In the standard tensor domain $S \otimes \texttt{SUM}$, these combine as $(a +_S b) \otimes 10$. If both paths are present ($a = b = 1_S$), the idempotency of S causes the result to collapse: $(1_S +_S 1_S) \otimes 10 = 1_S \otimes 10$. We therefore lose the information that the value 10 occurred twice, leading to an incorrect sum of 10 instead of 20.

In the bag-extended domain $S\mathbb{N} \otimes \texttt{SUM}$, the same result is represented as $(1 \cdot a + 1 \cdot b) \otimes 10$. By specialising this result via a homomorphism $f : S\mathbb{N} \to \mathbb{N}$ (mapping $a, b \mapsto 1$), the coefficient $1 \cdot a + 1 \cdot b$ evaluates to 2. The result then correctly becomes $2 \otimes 10$, which yields the expected sum of 20. ▽

We have thus far considered aggregation in a limited context: when it is the last operation of the query. In the following section, we generalise this framework to account for arbitrarily complex queries, where aggregation and grouping may appear as intermediate steps.

2. Arbitrary aggregation

Aggregation with grouping becomes challenging when the results may be fed into subsequent query operations, such as joins or selections. In this setting, the truth values of comparisons over such expressions are not determined a priori. To represent the outcomes of such operations, the symbolic expressions in $S \otimes M$ are extended with equality comparisons.

More specifically, we introduce to the semiring S new expressions of the form $[x = y]$ where $x, y \in S \otimes M$. If we need to compare an aggregate result with an $m \in M$, we use $\iota(m)$ instead. Intuitively, these expressions act as placeholders for the truth value of a comparison.

Example 4.2.1. If we perform a selection on the grouping result from Example 4.1.9 (analogous to a HAVING clause in SQL) for a price of 40, the resulting $(\mathcal{M}, \mathcal{S})$-relation would be as follows:

cust_id	price	
vsdjp	$o_1 \otimes 30$	$\delta(o_1) \cdot [o_1 \otimes 30 = 1 \otimes 40]$
bjvks	$o_2 \otimes 40$	$\delta(o_2) \cdot [o_2 \otimes 40 = 1 \otimes 40]$
hcbja	$o_3 \otimes 20 + o_4 \otimes 20$	$\delta(o_3 + o_4) \cdot [o_3 \otimes 20 + o_4 \otimes 20 = 1 \otimes 40]$

These equality expressions remain symbolic until the query is specialised via a homomorphism to a concrete semiring, at which point the comparisons can be replaced by 1 (if the equality holds) or 0 (otherwise). ▽

We now formalise the treatment of complex aggregate queries by constructing an extended structure.

Definition 4.2.2 (S^M). Let S be a commutative semiring. We define a semiring $\widehat{S} = \mathbb{N}[S \cup \{[c_1 = c_2] \mid c_1, c_2 \in S \otimes M\}]$, which treats equality expressions as additional variables. We then define S^M as the quotient of $\widehat{S}$ by a congruence relation $\sim$ satisfying the following identities for all $s_1, s_2 \in S$:

$$0_{\widehat{S}} \sim 0_S$$
$$1_{\widehat{S}} \sim 1_S$$
$$s_1 +_{\widehat{S}} s_2 \sim s_1 +_S s_2$$
$$s_1 \cdot_{\widehat{S}} s_2 \sim s_1 \cdot_S s_2$$

When $\iota : M \to S \otimes M$ is an isomorphism (with inverse h), we can resolve these equality expressions. In that case, we further impose the following identities for all $c_1, c_2 \in S \otimes M$:

$$[c_1 = c_2] \sim \begin{cases} 1_S & h(c_1) =_M h(c_2) \\ 0_S & \text{otherwise.} \end{cases}$$

▽

An important property of S^M is that whenever provenance is specialised to a concrete domain where $S \otimes M$ can be interpreted directly in M, these equality expressions collapse. We consider an example before presenting this result formally.

Example 4.2.3. Consider the customer `hcbja` from our previous example. The expression representing the selection criterion is initially:

$$[o_3 \otimes 20 + o_4 \otimes 20 = 1 \otimes 40]$$

Once we apply a homomorphism $g : \mathbb{N}[X] \to \mathbb{N}$ where $g(o_3) = g(o_4) = 1$, the expression evaluates to $[1 \otimes 40 = 1 \otimes 40]$. Therefore, the congruence rules replace the expression with $1_{\mathbb{N}}$, solving the selection and confirming the tuple's presence in the final result. ▽

Proposition 4.2.4. *If S and M are such that $S \otimes M$ and M are isomorphic via ι, then S^M and S are isomorphic.*

PROOF. Let e be an expression representing an element in S^M. We proceed by induction on the structure of e.

In the base case, e is either $s \in S$ or $[a = b]$ with $a, b \in S \otimes M$. In the former case, the first set of congruence rules directly identifies $s_{\widehat{S}}$ with s_S, thus $e \sim s \in S$. In the latter case, since ι is an isomorphism, there exists an inverse mapping $h : S \otimes M \to M$. By the second set of congruence rules, $[a = b]$ is identified with 1_S if $h(a) =_M h(b)$ and 0_S otherwise.

For the inductive step, let $e = e_1 + e_2$ or $e = e_1 \cdot e_2$. By the inductive hypothesis, there exist $s_1, s_2 \in S$ such that $e_1 \sim s_1$ and $e_2 \sim s_2$. Since $\sim$ is a congruence relation, it is compatible with the semiring operations: $e_1 + e_2 \sim s_1 +_S s_2$ and $e_1 \cdot e_2 \sim s_1 \cdot_S s_2$. As S is closed under addition and multiplication, the result of these operations remains in S.

Therefore, every expression in S^M matches to an element of S. Furthermore, since the construction of S^M preserves the faithful embedding of S, no two distinct elements of S are identified as equivalent. Thus, the mapping from S to S^M is a bijection. □

We now define a semantics for general aggregation queries. In this setting, every attribute value is an element of the tensor product $S^M \otimes M$. Standard values $m \in M$ are treated as elements of this domain via $\iota(m)$.

DEFINITION 4.2.5 ($(\mathcal{M}, \mathcal{S}^M)$-relational algebra). Let R, R_1, R_2 be $(\mathcal{M}, \mathcal{S}^M)$-relations over an attribute set U unless otherwise stated. The $(\mathcal{M}, \mathcal{S}^M)$-*relational algebra* operations are defined as follows:

- Union:

$$(R_1 \cup R_2)(t) = \begin{cases} \sum_{t' \in supp(R_1)} \left(R_1(t') \cdot \prod_{u \in U} [t'(u) = t(u)] \right) \quad + \\ \sum_{t'' \in supp(R_2)} \left(R_2(t'') \cdot \prod_{u \in U} [t''(u) = t(u)] \right) & t \in supp(R_1) \cup supp(R_2) \\ 0_S & \text{otherwise.} \end{cases}$$

- Projection: Let $U' \subseteq U$.

$$\pi_{U'}(R)(t) = \begin{cases} \sum_{t' \in supp(R)} \left(R(t') \cdot \prod_{u \in U'} [t(u) = t'(u)] \right) & t[U'] \in supp(R) \\ 0_S & \text{otherwise.} \end{cases}$$

- Selection: Let the equality condition θ involve attribute $u \in U$ and value $m \in M$.

$$(\sigma_\theta(R))(t) = R(t) \cdot [t(u) = \iota(m)]$$

- Join: Let R_1, R_2 have disjoint attribute sets U_1, U_2, and let the join be on $u_1 \in U_1$ and $u_2 \in U_2$.

$$(R_1 \bowtie_{R_1.u_1 = R_2.u_2} R_2)(t) = R_1(t[U_1]) \cdot R_2(t[U_2]) \cdot [t(u_1) = t(u_2)]$$

- Aggregation: Let R have a single attribute u.

$$AGG_M(R)(t) = \begin{cases} 1_S & t(u) = \sum_{t' \in supp(R)} (R(t') * t'(u)) \\ 0_S & \text{otherwise.} \end{cases}$$

- Grouping: Let $U' \subseteq U$ be the grouped attributes and $u \in U \setminus U'$ be the aggregated attribute, and let δ be naturally extended to S^M by preserving equality brackets.

$$GB_{U',u}(R)(t) = \begin{cases} \delta\big(\pi_{U'}(R)(t[U'])\big) & t(u) = \sum_{t' \in supp(R)} \left(R(t') \cdot \prod_{u' \in U'} [t'(u') = t(u')] \right) * t'(u) \\ 0_S & \text{otherwise.} \end{cases}$$

▽

Intuitively, we can identify three kinds of operations. First, union and projection, which traditionally merge identical tuples, must now account for values that are not yet determined. We ensure a tuple only contributes to the result if it is equivalent across all relevant attributes. This is enforced by multiplying the provenance by a conjunction of equality expressions.

Second, selection and join operations traditionally filter the relation based on data conditions. Because the truth of these conditions is unknown a priori, the operations retain all candidate tuples while multiplying their provenance by equality expressions.

Finally, aggregation and grouping utilise the semimodule structure to collect data values into formal sums, pairing each contributing tuple's provenance with its data value via the tensor product.

In essence, the operations record the formal conditions for a tuple's contribution via the equality expressions $[x = y]$. Each operation multiplies the provenance by these symbolic placeholders, which persist through the query even when data values are symbolic aggregates. These expressions resolve into values of 0 or 1 during specialisation.

We conclude this chapter by establishing the fundamental theorem for the $(\mathcal{M}, \mathcal{S}^M)$-relational algebra and provide an example of the framework in practice. We first formalise the process of lifting semiring homomorphisms on the domain $S^M \otimes M$.

Remark. Let $h : S \to S'$ be a δ-semiring homomorphism. We lift h to a transformation $\widehat{h}$ that acts on all expressions in the $(\mathcal{M}, \mathcal{S}^M)$ framework. This map is defined inductively:

- Base case: If $s \in S$, then $\widehat{h}(s) = h(s)$.
- Data domain: For an element in $S^M \otimes M$, then $\widehat{h}(\sum s_i \otimes m_i) = \sum \widehat{h}(s_i) \otimes m_i$.
- Equality brackets: For an equality bracket in the semiring S^M, then $\widehat{h}([x = y]) = [\widehat{h}(x) = \widehat{h}(y)]$.

The map h_{Rel} on $(\mathcal{M}, \mathcal{S}^M)$-relations is then defined by applying $\widehat{h}$ to every tuple annotation and every data value.

Theorem 4.2.6 (Fundamental theorem). *Let $h : S \to S'$ be a δ-semiring homomorphism. For any $(\mathcal{M}, \mathcal{S}^M)$-relational algebra query Q and any $(\mathcal{M}, \mathcal{S}^M)$-database D, the following identity holds:*

$$Q(h_{Rel}(D)) = h_{Rel}(Q(D)).$$

Proof. Assume that $h : S \to S'$ is a δ-semiring homomorphism. We show that $Q(h_{Rel}(D)) = h_{Rel}(Q(D))$ by induction on the structure of the query Q. The base case holds by definition. For the inductive step, we assume the statement holds for all sub-queries and examine the last applied operation.

- *Union.* Let $Q = Q_1 \cup Q_2$. By the definition of union, the induction hypothesis, and the fact that $\widehat{h}$ preserves equality brackets and semiring operations:

$$\begin{aligned}
Q(h_{Rel}(D))(t) &= \sum_{t'} \left(Q_1(h_{Rel}(D))(t') \cdot \prod_{u \in U} [\widehat{h}(t'(u)) = \widehat{h}(t(u))] \right) + \sum_{t''} \dots \\
&= \sum_{t'} \left(\widehat{h}(Q_1(D)(t')) \cdot \widehat{h}\left(\prod_{u \in U} [t'(u) = t(u)] \right) \right) + \sum_{t''} \dots \\
&= \widehat{h} \left(\sum_{t'} \left(Q_1(D)(t') \cdot \prod_{u \in U} [t'(u) = t(u)] \right) + \sum_{t''} \dots \right) \\
&= h_{Rel}(Q(D))(t).
\end{aligned}$$

- *Projection.* Let $Q = \pi_{U'}(Q')$. By the definition of projection, the induction hypothesis, and the preservation of equality brackets and semiring operations:

$$\begin{aligned} Q(h_{Rel}(D))(t) &= \sum_{t'} \left(Q'(h_{Rel}(D))(t') \cdot \prod_{u \in U'} [\widehat{h}(t(u)) = \widehat{h}(t'(u))] \right) \\ &= \sum_{t'} \left(\widehat{h}(Q'(D)(t')) \cdot \widehat{h}\left(\prod_{u \in U'} [t(u) = t'(u)] \right) \right) \\ &= \widehat{h}\left(\sum_{t'} \left(Q'(D)(t') \cdot \prod_{u \in U'} [t(u) = t'(u)] \right) \right) \\ &= h_{Rel}(Q(D))(t). \end{aligned}$$

- *Selection.* Let $Q = \sigma_\theta(Q')$ with θ equating attribute $u \in U$ with value $m \in M$. By the definition of selection, the induction hypothesis, and the preservation properties of $\widehat{h}$:

$$\begin{aligned} Q(h_{Rel}(D))(t) &= Q'(h_{Rel}(D))(t) \cdot [\widehat{h}(t(u)) = \widehat{h}(\iota(m))] \\ &= \widehat{h}(Q'(D)(t)) \cdot \widehat{h}([t(u) = \iota(m)]) \\ &= \widehat{h}(Q'(D)(t) \cdot [t(u) = \iota(m)]) \\ &= h_{Rel}(Q(D))(t). \end{aligned}$$

- *Join.* Let $Q = Q_1 \bowtie Q_2$. By the definition of join, the induction hypothesis, and the preservation properties of $\widehat{h}$:

$$\begin{aligned} Q(h_{Rel}(D))(t) &= Q_1(h_{Rel}(D))(t[U_1]) \cdot Q_2(h_{Rel}(D))(t[U_2]) \cdot [\widehat{h}(t(u_1)) = \widehat{h}(t(u_2))] \\ &= \widehat{h}(Q_1(D)(t[U_1])) \cdot \widehat{h}(Q_2(D)(t[U_2])) \cdot \widehat{h}([t(u_1) = t(u_2)]) \\ &= \widehat{h}(Q_1(D)(t[U_1]) \cdot Q_2(D)(t[U_2]) \cdot [t(u_1) = t(u_2)]) \\ &= h_{Rel}(Q(D))(t). \end{aligned}$$

- *Aggregation.* Let $Q = AGG_M(Q')$. The value is $v = \sum s_i * m_i$ and annotation is 1_S. Applying h_{Rel} to this result transforms the annotation to $\widehat{h}(1_S) = 1_{S'}$ and the value to $\widehat{h}(v) = \sum \widehat{h}(s_i) * (m_i)$. Conversely, by the induction hypothesis, this sub-query result already contains the transformed annotations $\widehat{h}(s_i)$ and values m_i. Summing these directly yields the same result with annotation $1_{S'}$.

- *Grouping.* Let $Q = GB_{U',u}(Q')$. For the annotation, applying h_{Rel} to the grouping result yields $\widehat{h}(\delta(\pi_{U'}(Q')(D)))$. Since $\widehat{h}$ and δ both preserve equality brackets and semiring operations, they commute, transforming the expression to $\delta(\widehat{h}(\pi_{U'}(Q')(D)))$. Conversely, by the induction hypothesis for projection, $\widehat{h}(\pi_{U'}(Q')(D))$ is exactly the annotation produced by $\pi_{U'}(Q'(h_{Rel}(D)))$, so applying δ to it confirms the identity. The aggregated data values commute by the same argument used for aggregation.

□

Example 4.2.7. If we take the output of Example 4.2.1 and apply a further aggregation, the resulting value v is:

$$\begin{aligned} (\delta(o_1) \cdot [o_1 \otimes 30 = 1 \otimes 40]) * (o_1 \otimes 30) \quad + \\ (\delta(o_2) \cdot [o_2 \otimes 40 = 1 \otimes 40]) * (o_2 \otimes 40) \quad + \\ (\delta(o_3 + o_4) \cdot [o_3 \otimes 20 + o_4 \otimes 20 = 1 \otimes 40]) * (o_3 \otimes 20 + o_4 \otimes 20) \end{aligned}$$

▽

As queries grow in complexity, the resulting provenance expressions become increasingly difficult to read. Provenance query languages address this by treating these expressions as input and allowing users to filter and summarise the raw provenance [Kar+10].

CHAPTER 5

Difference

A notable omission from the standard semiring provenance framework is the difference (or negation) operation. By definition, a semiring provides structure for addition and multiplication, but it does not require the existence of inverses. Consequently, there is no canonical method to incorporate difference.

Extending the framework to include difference while respecting the underlying semantics is a challenging task. In bag semantics, for instance, standard integer subtraction is problematic because it can yield negative multiplicities, which are not defined for bags. Furthermore, a rigorous treatment of difference requires the provenance to account not only for the presence of a tuple, but also for its absence.

Given these challenges, several competing approaches have been proposed, and their respective merits remain a subject of scientific debate. Suciu [Suc24] identifies three ways this has been attempted: by the introduction of additional algebraic identities, the use of a semiring's natural order, and the construction of quotient semirings.

The most direct approach to incorporating difference is to extend the semiring by introducing additional identities. Given any set of desired identities, there exists a unique way to freely extend a semiring S to a new structure that satisfies them. However, the challenge lies in selecting a set of identities that provides a meaningful notion of difference without compromising the integrity of the original semiring's structure.

A candidate for these identities is the set of ring axioms. By requiring every element a to have an additive inverse $-a$ such that $a + (-a) = 0$, we transition from semirings to rings. This works for some structures like the semiring of natural numbers, which extends to the ring of integers, assuming we define the meaning of a negative multiplicity. However, it leads to a collapse in others. More specifically, any idempotent semiring becomes trivial when forced to satisfy ring axioms [Suc24].

To avoid issues such as negative multiplicities and structural collapse, researchers have explored more nuanced ways to formalise difference. Without relying on full additive inverses, Amsterdamer, Deutch, and Tannen [Ams+11a] formulate a set of axioms (see Figure 6) that any difference operation should satisfy, and investigate whether the semiring framework can be extended to meet these requirements regardless of the underlying semiring.

Amsterdamer et al. consider a number of candidates but find that all of them fail to satisfy the complete set of (A1)–(A5). While we could ensure these identities by constructing the free algebraic structure satisfying the commutative semiring axioms and (A1)–(A5), the authors find this structure quite uninformative.

$$
\begin{array}{ll}
\text{(A1)} & a - a = 0 \\
\text{(A2)} & 0 - a = 0 \\
\text{(A3)} & a + (b - a) = b + (a - b) \\
\text{(A4)} & a - (b + c) = (a - b) - c \\
\text{(A5)} & a \cdot (b - c) = a \cdot b - a \cdot c
\end{array}
$$

FIGURE 6. Axioms for difference

This suggests that a compromise between the practicality of such an extension and the adherence to the algebraic identities is needed to realise a difference operation.

1. Natural order

An alternative to the algebraic identity approach utilises the inherent order of a semiring. Geerts and Poggi [GP10] focus on a class of semirings that possess a natural order, where the notion of difference is derived from the structural relationship between elements rather than from additive inverses.

We begin by defining the natural order for commutative monoids and its extension to semirings.

Definition 5.1.1 (Natural order). Given a commutative monoid $(M, +, 0)$, the *natural preorder* $\preccurlyeq$ is defined as follows:

$$a \preccurlyeq b \iff \exists z \in M : a + z = b.$$

This relation is always reflexive and transitive. If it is also antisymmetric ($a \preccurlyeq b$ and $b \preccurlyeq a \implies a = b$), then it is a partial order and we call it the *natural order*.

A semiring is *naturally ordered* if the natural preorder of its additive monoid is a partial order. ▽

Example 5.1.2. Many of the semirings of interest are naturally ordered:

- In $\mathbb{N}$, the natural order $\preccurlyeq$ coincides with the standard $\leq$ relation.
- In $\mathbb{B}$, the natural order is $0 \preccurlyeq 1$, since $0 + 1 = 1$.
- In $\mathbb{N}[X]$, the natural order is determined by the coefficients of each monomial: for any polynomials P and Q, $P \preccurlyeq Q$ if and only if, for every monomial m, the coefficient of m in P is less than or equal to the corresponding coefficient in Q.

In contrast, $(\mathbb{Z}, +, \cdot, 0, 1)$ is not naturally ordered. Because every element has an additive inverse, for any distinct $a, b \in \mathbb{Z}$, we always have $a \preccurlyeq b$ (by choosing $z = b - a$). Since $a \preccurlyeq b$ and $b \preccurlyeq a$ but not $a = b$, antisymmetry fails and the relation is not a partial order. ▽

In a naturally ordered monoid, the difference between two elements can be captured by seeking the least element that bridges the gap between them. We introduce monus, the operation that formalises this notion.

Definition 5.1.3 (Monus). Let $(M, +, 0)$ be a naturally ordered commutative monoid. For any two elements $a, b \in M$, the *monus* of a and b, denoted by $a \ominus b$, is the smallest element $z \in M$ such that $a \preccurlyeq b + z$, provided such an element exists. ▽

In general, the existence of a monus is not guaranteed for all pairs in a naturally ordered monoid. To ensure that monus is always well-defined, we focus on structures where this operation is total.

Definition 5.1.4 (Commutative monoid with monus). A naturally ordered commutative monoid is called a *commutative monoid with monus* (CMM) if the monus $a \ominus b$ exists for every pair of elements $a, b \in M$. ▽

Definition 5.1.5 (M-semiring). A semiring is called an *m-semiring* (or a semiring with monus) if its additive monoid is a CMM. We can represent an m-semiring by the 6-tuple $(S, +_S, \cdot_S, 0_S, 1_S, \ominus_S)$.

If S and S' are m-semirings, a homomorphism between them is a semiring homomorphism $h : S \to S'$ such that h preserves monus, i.e. $h(x \ominus_S y) = h(x) \ominus_{S'} h(y)$. ▽

Example 5.1.6. The natural numbers form an m-semiring by defining monus as:

$$a \ominus b = \begin{cases} a - b & \text{if } a \geq b, \\ 0 & \text{otherwise.} \end{cases}$$

This extends to the polynomial semiring where monus is applied on the coefficients of each monomial. For the Boolean semiring, monus can also be defined casewise: $a \ominus b = 1$ if $a = 1$ and $b = 0$, and 0 otherwise.

Lastly, we can define monus in the tropical semiring as:

$$a \ominus b = \begin{cases} \infty & \text{if } a \geq b \\ a & \text{otherwise.} \end{cases}$$

▽

Having established the notion of an m-semiring, we now extend the $\mathcal{S}$-relational algebra with a difference operation.

Definition 5.1.7 (Difference). Let $\mathcal{S}$ be an m-semiring and R_1, R_2 be $\mathcal{S}$-relations. We extend the $\mathcal{S}$-relational algebra with a difference operation defined as:

$$(R_1 - R_2)(t) = R_1(t) \ominus R_2(t)$$

▽

We then lift an m-semiring homomorphism h to the level of relations and databases by applying h to each tuple annotation so that we can establish the fundamental theorem. Since the proof follows the same pattern, the details are omitted and the reader is referred to the proof provided by Geerts and Poggi [GP10].

Theorem 5.1.8 (Fundamental theorem). *Assume that S, S' are m-semirings and let $h : S \to S'$ be a map. The identity $Q(h_{Rel}(D)) = h_{Rel}(Q(D))$ holds for every $\mathcal{S}$-relational algebra query Q and every $\mathcal{S}$-database D if and only if h is an m-semiring homomorphism.*

We next examine the structure that serves as the provenance domain for m-semirings. Although the polynomial semiring is an m-semiring, it is not the free m-semiring. We illustrate this with an example:

Example 5.1.9. Consider a relation R containing a single tuple t, and the query $Q = (R \bowtie R) - R$. Let the tuple t be annotated with the value 2 in the bag semiring. The query evaluation yields:

$$(2 \cdot 2) \ominus 2 = 4 \ominus 2 = 2.$$

Now, consider the same tuple annotated with the variable x. In the polynomial semiring, the symbolic evaluation yields:

$$(x \cdot x) \ominus x = x^2 \ominus x.$$

To compute this monus, we align the coefficients of corresponding monomials in $\mathbb{N}[X]$ as follows:

$$x^2 = 1 \cdot x^2 + 0 \cdot x$$
$$x = 0 \cdot x^2 + 1 \cdot x$$

As an m-semiring, $\mathbb{N}[X]$ performs the monus operation by applying truncated subtraction to each pair of coefficients. We obtain $x^2 \ominus x = 1 \cdot x^2 + 0 \cdot x = x^2$. If we apply a valuation that maps $x \mapsto 2$, we get an annotation of 4 instead of 2 as above. ▽

Fortunately, the free m-semiring can be constructed by leveraging the identities that define the class of CMMs. Amer [Ame84] established an elegant characterisation of CMMs, and consequently of m-semirings, through a concise set of identities, for which a proof is provided in [Suc24].

Theorem 5.1.10. *A commutative monoid is a CMM if and only if it satisfies (A1)–(A4):*

$$
\begin{aligned}
&\text{(A1)} \quad a - a = 0 \\
&\text{(A2)} \quad 0 - a = 0 \\
&\text{(A3)} \quad a + (b - a) = b + (a - b) \\
&\text{(A4)} \quad a - (b + c) = (a - b) - c
\end{aligned}
$$

To construct the free m-semiring generated by a set of variables X, we follow the standard process:

We consider the set of all possible terms that can be formed using the variables in X, the constants 0 and 1, and the operations of addition, multiplication, and monus.

We then define a congruence relation that groups together any two terms that can be proven equal using the commutative semiring axioms and (A1)–(A4).

The resulting quotient structure is the free m-semiring because it satisfies the required axioms without assuming any additional properties. This ensures that any assignment of variables to values from an m-semiring S can be uniquely extended to a homomorphism from the free m-semiring to S.

One shortcoming of m-semirings is that the monus operation relies exclusively on the additive monoid and does not account for the multiplicative monoid. Consequently, as noted by Amsterdamer et al. [Ams+11a], the distributive identity (A5) is not guaranteed to hold across all m-semirings.

Despite this limitation, Badia, Kolaitis, and Noguera [Bad+25] defend the m-semiring approach (and go on to prove Codd's theorem for databases over m-semirings), arguing that this is an acceptable trade-off. They justify this stance on three grounds.

Firstly, the class of m-semirings remains sufficiently broad, encompassing most of the important semirings. Secondly, they believe that the failure of certain identities within m-semirings should not be treated as problematic, given that arbitrary commutative semirings already lack other fundamental identities. Finally, and perhaps most importantly from a practical standpoint, the m-semiring axioms faithfully reflect the semantics of the difference operation in SQL.

2. Quotient constructions

A third approach to modelling difference involves the use of quotient constructions. Unlike methods that rely on the intrinsic properties of a semiring, this approach provides a way to engineer the desired algebraic behaviour. The core idea is to construct a larger domain of provenance expressions and subsequently impose a congruence relation to identify which expressions should be considered identical. By carefully defining this equivalence, we can control the interaction between the presence and absence of data, refining the provenance domain to support difference while respecting the structural integrity of the underlying semiring.

An important contribution to this line of research comes from Grädel and Tannen [GT25], who developed a semiring semantics for first-order logic. They achieve this by constructing quotient semirings over polynomials with dual indeterminates, where each atomic fact is paired with a corresponding dual element representing its absence.

While various extensions have attempted to model the difference of relations, they have often failed to provide a framework for tracking absent or negative information. To address this, Grädel and Tannen formalise a semantics based on literals rather than positive facts alone.

Let τ be a finite relational vocabulary and A be a finite, non-empty universe. From these, we construct the set $\mathrm{Facts}_A(\tau)$ of all ground relational atoms and the set $\mathrm{NegFacts}_A(\tau)$ of their negations. The union of these sets forms the set of literals over τ and A: $\mathrm{Lit}_A(\tau) = \mathrm{Facts}_A(\tau) \cup \mathrm{NegFacts}_A(\tau)$.

We utilise a commutative semiring $\mathcal{S} = (S, +, \cdot, 0, 1)$ to interpret these assertions. The element 0 represents false assertions, while any nonzero element provides an annotated interpretation of true assertions.

The semantics is formalised through an $\mathcal{S}$-interpretation, which maps literals to elements of the semiring, and is then extended to all first-order formulae. Because the semiring does not provide a direct mechanism for negation, we adopt the syntactic transformation to negation normal form (NNF), denoted by $\varphi \mapsto \mathrm{nnf}(\varphi)$, which ensures that negations only appear at the level of literals.

Definition 5.2.1 ($\mathcal{S}$-interpretation). An *$\mathcal{S}$-interpretation* is a mapping $\pi : \mathrm{Lit}_A(\tau) \to S$. This mapping induces an *$\mathcal{S}$-valuation* $\pi[\![\varphi(\vec{a})]\!]$ for any first-order formula $\varphi(\vec{x}) \in \mathrm{FO}(\tau)$ and instantiation $\vec{a} \subseteq A$. We first extend π by treating atomic equalities: for any distinct $a, b \in A$, we have $\pi[\![a = b]\!] := 0$ and $\pi[\![a = a]\!] := 1$ (and analogously for inequalities). For compound formulae:

$$\pi[\![\psi \vee \varphi]\!] := \pi[\![\psi]\!] + \pi[\![\varphi]\!] \qquad \pi[\![\psi \wedge \varphi]\!] := \pi[\![\psi]\!] \cdot \pi[\![\varphi]\!]$$
$$\pi[\![\exists x\, \psi(x)]\!] := \sum_{a \in A} \pi[\![\psi(a)]\!] \qquad \pi[\![\forall x\, \psi(x)]\!] := \prod_{a \in A} \pi[\![\psi(a)]\!]$$
$$\pi[\![\neg\varphi]\!] := \pi[\![\mathrm{nnf}(\neg\varphi)]\!]$$

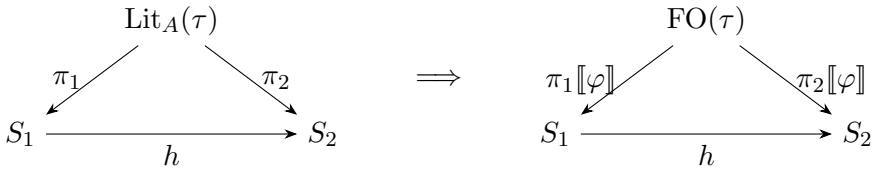
▽

Note that the provenance of equality and inequality atoms is not tracked, because they are interpreted as 1 or 0.

To prove the fundamental theorem within this framework, we must first address the challenge introduced by the presence of negation. We can see that the valuation of a formula remains the same under the transformation to negation normal form. Therefore it suffices to consider formulae where negations have been pushed down to the level of atomic literals.

Proposition 5.2.2. $\pi[\![\varphi]\!] = \pi[\![\mathrm{nnf}(\varphi)]\!]$.

Theorem 5.2.3 (Fundamental theorem). *Let $h : \mathcal{S}_1 \to \mathcal{S}_2$ be a semiring homomorphism and let $\pi_1 \colon \mathrm{Lit}_A(\tau) \to S_1$ and $\pi_2 \colon \mathrm{Lit}_A(\tau) \to S_2$ be interpretations such that $h \circ \pi_1 = \pi_2$. Then, for every sentence $\varphi \in \mathrm{FO}(\tau)$, we have $h(\pi_1[\![\varphi]\!]) = \pi_2[\![\varphi]\!]$. As diagrams:*



Proof. By Proposition 5.2.2, any first-order formula φ is semantically equivalent to its negation normal form under any $\mathcal{S}$-interpretation. It therefore suffices to prove $h(\pi_1[\![\varphi]\!]) = \pi_2[\![\varphi]\!]$ for formulae constructed using $\vee, \wedge, \exists$, and $\forall$.

Assume $h : \mathcal{S}_1 \to \mathcal{S}_2$ is a semiring homomorphism and $h \circ \pi_1 = \pi_2$. We proceed by induction on the structure of φ:

In the base case, φ is either a literal or an equality/inequality atom. For literals, by the definition of an $\mathcal{S}$-valuation:

$$\pi_1[\![L]\!] = \pi_1(L) \quad \text{and} \quad \pi_2[\![L]\!] = \pi_2(L)$$

Applying the homomorphism h and the assumption $h \circ \pi_1 = \pi_2$:

$$h(\pi_1[\![L]\!]) = h(\pi_1(L)) = (h \circ \pi_1)(L) = \pi_2(L) = \pi_2[\![L]\!]$$

For equality/inequality atoms, the identity holds because $\pi[\![a = a]\!] = 1$ and $\pi[\![a = b]\!] = 0$, and semiring homomorphisms must satisfy $h(1) = 1$ and $h(0) = 0$.

For the inductive step, we assume the identity holds for sub-formulae and perform a case analysis on φ:

Disjunction. Let $\varphi = \psi \vee \chi$. By the definition of $\mathcal{S}$-valuation, the fact that h preserves addition, and the induction hypothesis:

$$\begin{aligned} h(\pi_1[\![\psi \vee \chi]\!]) &= h(\pi_1[\![\psi]\!] + \pi[\![\chi]\!]) \\ &= h(\pi_1[\![\psi]\!]) + h(\pi_1[\![\chi]\!]) \\ &= \pi_2[\![\psi]\!] + \pi_2[\![\chi]\!] \\ &= \pi_2[\![\psi \vee \chi]\!] \end{aligned}$$

Conjunction. Let $\varphi = \psi \wedge \chi$. By the definition of $\mathcal{S}$-valuation, the fact that h preserves multiplication, and the induction hypothesis:

$$\begin{aligned} h(\pi_1[\![\psi \wedge \chi]\!]) &= h(\pi_1[\![\psi]\!] \cdot \pi_1[\![\chi]\!]) \\ &= h(\pi_1[\![\psi]\!]) \cdot h(\pi_1[\![\chi]\!]) \\ &= \pi_2[\![\psi]\!] \cdot \pi_2[\![\chi]\!] \\ &= \pi_2[\![\psi \wedge \chi]\!] \end{aligned}$$

Existential Quantification. Let $\varphi = \exists x \psi(x)$. By the definition of $\mathcal{S}$-valuation, the fact that h preserves finite sums, and the induction hypothesis:

$$\begin{aligned} h(\pi_1[\![\exists x \psi(x)]\!]) &= h\left(\sum_{a \in A} \pi_1[\![\psi(a)]\!]\right) \\ &= \sum_{a \in A} h(\pi_1[\![\psi(a)]\!]) \\ &= \sum_{a \in A} \pi_2[\![\psi(a)]\!] \\ &= \pi_2[\![\exists x \psi(x)]\!] \end{aligned}$$

Universal Quantification. Let $\varphi = \forall x \psi(x)$. By the definition of $\mathcal{S}$-valuation, the fact that h preserves finite products, and the induction hypothesis:

$$\begin{aligned} h(\pi_1[\![\forall x \psi(x)]\!]) &= h\left(\prod_{a \in A} \pi_1[\![\psi(a)]\!]\right) \\ &= \prod_{a \in A} h(\pi_1[\![\psi(a)]\!]) \\ &= \prod_{a \in A} \pi_2[\![\psi(a)]\!] \\ &= \pi_2[\![\forall x \psi(x)]\!] \end{aligned}$$

□

As noted above, a valuation of $\pi[\![\varphi]\!] = 0$ indicates that φ is false under the interpretation of π, while a nonzero valuation $\pi[\![\varphi]\!] = s$ signifies that φ is true with annotation s. We now formalise the conditions under which such an interpretation defines a model.

Definition 5.2.4 (Model-defining interpretation). An $\mathcal{S}$-interpretation $\pi : \mathrm{Lit}_A(\tau) \to S$ is *model-defining* if, for each atom $R(\vec{a})$, exactly one of the values $\pi(R(\vec{a}))$ and $\pi(\neg R(\vec{a}))$ is 0. Consequently, every model-defining interpretation π uniquely determines a model $\mathfrak{A}_\pi$ such that for any literal L we have $\mathfrak{A}_\pi \vDash L$ if and only if $\pi(L) \neq 0$. ▽

While a model-defining interpretation allows us to extract a model $\mathfrak{A}_\pi$ from an $\mathcal{S}$-interpretation, we can also perform the converse construction. Given any model $\mathfrak{A}$, we can define an interpretation that recovers standard model-theoretic truth.

Example 5.2.5. Let $\mathfrak{A}$ be a model over the relational vocabulary τ with universe A. The *canonical truth interpretation* is the model-defining interpretation $\pi_{\mathfrak{A}} : \mathrm{Lit}_A(\tau) \to \mathbb{B}$ defined by:

$$\pi_{\mathfrak{A}}(L) = \begin{cases} 1 & \text{if } \mathfrak{A} \models L \\ 0 & \text{otherwise.} \end{cases}$$

Under this interpretation, $\pi_{\mathfrak{A}}[\![\varphi]\!] = 1$ if and only if $\mathfrak{A} \models \varphi$. ▽

To justify the choice of semirings as the semantic domain, Grädel and Tannen provide a proof-theoretic semantics. The valuation of a formula in a semiring is then shown to be exactly the sum of the valuations of its proof trees.

Definition 5.2.6 (Valuation tree). A *valuation tree* for a sentence $\psi \in \mathrm{FO}$ and an $\mathcal{S}$-interpretation $\pi : \mathrm{Lit}_A(\tau) \to S$ is a directed tree $\mathcal{T}$, whose nodes are labelled by occurrences of formulae $\varphi(\vec{a})$, where $\varphi(\vec{x})$ is a subformula of ψ whose free variables $\vec{x}$ are instantiated by $\vec{a} \subseteq A$, such that the following conditions hold:

- The root of $\mathcal{T}$ is ψ.
- A node $\varphi \vee \vartheta$ has one child which is either φ or ϑ.
- A node $\varphi \wedge \vartheta$ has two children φ and ϑ.
- A node $\exists y\, \varphi(\vec{a}, y)$ has one child $\varphi(\vec{a}, b)$ for some $b \in A$.
- A node $\forall y\, \varphi(\vec{a}, y)$ has the children $\varphi(\vec{a}, b)$ for all $b \in A$.
- A leaf of $\mathcal{T}$ is a literal or an equality/inequality atom.

For any literal L, let $\#_L(\mathcal{T})$ be the number of occurrences of L in $\mathcal{T}$. The valuation of $\mathcal{T}$ is

$$\pi(\mathcal{T}) := \prod_{L \in \mathrm{Lit}_A(\tau)} \pi(L)^{\#_L(\mathcal{T})}.$$

▽

Definition 5.2.7 (Proof tree). A *proof tree* for π and $\psi \in \mathrm{FO}$ is a valuation tree $\mathcal{T}$ with $\pi(\mathcal{T}) \neq 0$. If π is clear from the context, we write $T(\psi)$ for the set of all proof trees for π and ψ. ▽

The equivalence between the valuation of a formula and its proof trees is then formalised by the following theorem, for which a proof is provided in [GT25]:

Theorem 5.2.8. *For every $\mathcal{S}$-interpretation $\pi : \mathrm{Lit}_A(\tau) \to S$ and every sentence $\psi \in \mathrm{FO}(\tau)$:*

$$\pi([\![\psi]\!]) = \sum_{\mathcal{T} \in T(\psi)} \pi(\mathcal{T}).$$

We now construct the provenance domain. We use a variation of the concept of polynomials to handle negated facts. The main insight is the use of indeterminates (variables) in positive-negative pairs.

Let $X, \overline{X}$ be two disjoint sets with a one-to-one correspondence between them. We denote by $p \in X$ and $\overline{p} \in \overline{X}$ two elements that are in this correspondence. We refer to the elements of $X \cup \overline{X}$ as *provenance tokens*.

We fix a finite non-empty set A and consider $\mathrm{Lit}_A(\tau)$. We annotate $\mathrm{Facts}_A(\tau)$ using X and, likewise, we annotate $\mathrm{NegFacts}_A(\tau)$ using $\overline{X}$. By convention, if we annotate $R(\vec{a})$ with the positive token p, then the negative token $\overline{p}$ can only be used to annotate $\neg R(\vec{a})$, and vice versa. We refer to p and $\overline{p}$ as *complementary tokens*.

Definition 5.2.9 (Dual-indeterminate polynomials). We denote by $\mathbb{N}[X, \overline{X}]$ the quotient of the semiring of polynomials $\mathbb{N}[X \cup \overline{X}]$ by the congruence $p \cdot \overline{p} \sim 0$ for all $p \in X$. We call the elements of $\mathbb{N}[X, \overline{X}]$ *dual-indeterminate* polynomials. ▽

The congruence relation ensures that any monomials (proof trees) that contain complementary (contradictory) tokens are eliminated from the provenance. Hence, the congruence classes in $\mathbb{N}[X, \overline{X}]$ are in one-to-one correspondence with the polynomials in $\mathbb{N}[X \cup \overline{X}]$ whose monomials do not contain complementary tokens. For example, $x \cdot z$ is equivalent to $x \cdot z + (y \cdot \overline{y} \cdot w)$ in $\mathbb{N}[X, \overline{X}]$.

We now consider interpretations into the semiring of dual-indeterminate polynomials.

Definition 5.2.10 (Provenance-tracking interpretation). A *provenance-tracking* interpretation is an $\mathbb{N}[X, \overline{X}]$-interpretation $\pi : \mathrm{Lit}_A(\tau) \to \mathbb{N}[X, \overline{X}]$ such that $\pi(\mathrm{Facts}_A(\tau)) \subseteq X \cup \{0, 1\}$ and $\pi(\mathrm{NegFacts}_A(\tau)) \subseteq \overline{X} \cup \{0, 1\}$. ▽

To track provenance for a given model we use an interpretation that is both model-defining and provenance-tracking. The idea is that if the interpretation annotates a positive or negative atom with a token, then we wish to track that atom. On the other hand annotating with 0 or 1 is done when we do not track the atom, yet we need to recall whether it holds in the model.

Example 5.2.11. Consider a social network with universe $A = \{a, b, c\}$ (Alice, Bob, Charlie). We use $F(x, y)$ to denote a relation where x follows y. Suppose we are interested in the sentence φ which asserts that there is a user whom both Alice and Bob follow, but who does not follow Alice back:

$$\varphi := \exists z(F(a, z) \wedge F(b, z) \wedge \neg F(z, a))$$

We define the following provenance-tracking interpretation $\beta : \mathrm{Lit}_A \to \mathbb{N}[X, \overline{X}]$:

- $\beta(F(a, c)) = p$, $\beta(F(b, c)) = q$, $\beta(\neg F(c, a)) = \overline{r}$
- $\beta(F(a, b)) = s$, $\beta(F(b, b)) = 1$, $\beta(\neg F(b, a)) = 1$
- All other literals are 0 (if positive) or 1 (if negative).

The provenance polynomial for φ is calculated as:

$$\begin{aligned} \beta[\![\varphi]\!] &= \sum_{z \in \{a,b,c\}} \beta(F(a, z)) \cdot \beta(F(b, z)) \cdot \beta(\neg F(z, a)) \\ &= (0 \cdot 0 \cdot 1) + (s \cdot 1 \cdot 1) + (p \cdot q \cdot \overline{r}) \\ &= s + pq\overline{r} \end{aligned}$$

meaning that the monomial s is a proof tree (because Alice follows Bob, Bob follows himself, but Bob does not follow Alice) in addition to $pq\overline{r}$ (because Alice follows Charlie, Bob follows Charlie, but Charlie does not follow Alice).

Furthermore, we can use the Viterbi semiring $\mathbb{V}$ to compute a confidence score for φ. We map the provenance tokens to confidence scores $f : X \cup \overline{X} \to [0, 1]$:

- $f(p) = 0.9, f(q) = 0.8, f(\overline{r}) = 0.7, f(s) = 0.4$
- For any other $x \in X$, $f(x) = 0$ and $f(\overline{x}) = 1$.

By the universal property of the dual-indeterminate polynomials, we can extend f to a homomorphism $h : \mathbb{N}[X, \overline{X}] \to \mathbb{V}$. Now, using the fundamental theorem, we can evaluate the provenance polynomial for φ in $\mathbb{V}$: $h(\beta[\![\varphi]\!]) = \max(f(s), f(p) \cdot f(q) \cdot f(\overline{r})) = \max(0.4, 0.504) = 0.504$. ▽

While model-defining interpretations provide a method for analysing a fixed structure, they are inherently limited by their focus on a single, predetermined model. A more versatile

analysis is possible when we consider provenance-tracking interpretations that are not model-defining. By allowing the interpretation to remain open regarding which facts hold, we can evaluate a formula over a class of potential models simultaneously.

This forms the basis of *reverse provenance analysis*, where the resulting symbolic expressions can be solved to identify specific models that satisfy particular desiderata. This technique is especially useful for applications such as explaining missing query results or determining the minimal updates required to repair an inconsistent database.

Definition 5.2.12 (Model-compatible interpretation). A provenance-tracking interpretation $\pi : \mathrm{Lit}_A(\tau) \to \mathbb{N}[X, \overline{X}]$ is *model-compatible* if for each atom $R(\vec{a})$ exactly one of the following three properties holds:

(1) There exists a token $z \in X$ such that $\pi(R(\vec{a})) = z$ and $\pi(\neg R(\vec{a})) = \overline{z}$, or

(2) $\pi(R(\vec{a})) = 0$ and $\pi(\neg R(\vec{a})) = 1$, or

(3) $\pi(R(\vec{a})) = 1$ and $\pi(\neg R(\vec{a})) = 0$

▽

Unlike model-defining interpretations, a model-compatible interpretation does not correspond to a single, fixed structure. Instead, it defines a family of potential models by allowing the truth of certain atoms to remain indeterminate (represented by the complementary tokens z and $\overline{z}$) while fixing others as semiring constants.

Model-compatible interpretations therefore serve as generalisations of model-defining interpretations: a specific model is obtained only when each pair of complementary tokens is specialised to a concrete truth value, obtaining a single structure from the family of compatible models.

To make this relationship precise, we define what it means for a specific structure to be compatible with such an interpretation.

Definition 5.2.13 (Compatible). Let $\pi\colon \mathrm{Lit}_A(\tau) \to \mathbb{N}[X, \overline{X}]$ be a model-compatible interpretation and let $\mathfrak{A}$ be a structure with universe A. We say that $\mathfrak{A}$ is *compatible* with π if $\mathfrak{A} \models L$ for every literal $L \in \mathrm{Lit}_A(\tau)$ such that $\pi(L) = 1$. ▽

Evaluating a query under a model-compatible interpretation produces a polynomial that serves as a catalogue of the conditions required for truth across the entire family of compatible structures. By selecting a particular monomial, we can identify a concrete model from the family that yields the desired outcome, enabling us to work backwards from a query result to the specific database that produces it. We end the chapter by applying reverse provenance analysis to our social network example.

Example 5.2.14. Continuing Example 5.2.11, suppose we are interested in the sentence ψ which states that no one is followed by everyone:

$$\psi := \forall y \exists x \neg F(x, y)$$

We define a model-compatible interpretation π and calculate $\pi[\![\psi]\!]$:

$$\begin{aligned}
\pi(F(b,a)) &= t, & \pi(\neg F(b,a)) &= \overline{t} \\
\pi(F(c,a)) &= r, & \pi(\neg F(c,a)) &= \overline{r} \\
\pi(F(a,b)) &= s, & \pi(\neg F(a,b)) &= \overline{s} \\
\pi(F(c,b)) &= w, & \pi(\neg F(c,b)) &= \overline{w} \\
\pi(F(a,c)) &= p, & \pi(\neg F(a,c)) &= \overline{p} \\
\pi(F(b,c)) &= q, & \pi(\neg F(b,c)) &= \overline{q} \\
\text{For all } x \in A, \pi(F(x,x)) &= 1, & \pi(\neg F(x,x)) &= 0.
\end{aligned}$$

$$\begin{aligned}\pi[\![\psi]\!] &= \prod_{y\in A}\sum_{x\in A}\pi(\neg F(x,y)) \\ &= (0+\overline{t}+\overline{r})\cdot(\overline{s}+0+\overline{w})\cdot(\overline{p}+\overline{q}+0) \\ &= (\overline{t}+\overline{r})(\overline{s}+\overline{w})(\overline{p}+\overline{q}) \\ &= \overline{t}\overline{s}\overline{p}+\overline{t}\overline{s}\overline{q}+\overline{t}\overline{w}\overline{p}+\overline{t}\overline{w}\overline{q}+\overline{r}\overline{s}\overline{p}+\overline{r}\overline{s}\overline{q}+\overline{r}\overline{w}\overline{p}+\overline{r}\overline{w}\overline{q}\end{aligned}$$

▽

While elegant and comprehensive, a critique of this approach is that the transformation to negation normal form is a syntactic restriction that flattens nested structure [Bad+25]. This restriction means that provenance evaluation is not compositional with respect to negation: the provenance of a negated formula, such as $\pi[\![\neg(\psi\wedge\chi)]\!]$, cannot be computed directly from $\pi[\![\psi\wedge\chi]\!]$. Instead, the entire formula must be syntactically rewritten prior to evaluation, which destroys the nested structure of the original query and prevents us from computing provenance as the query is evaluated.

CHAPTER 6

Conclusion

We have now reached the end of our investigation of semiring-based provenance. The study began with the foundational results of Green et al. [Gre+07] and moved to extensions of the semiring framework for more expressive queries. Amsterdamer et al.'s paper [Ams+11b] served as the main reference for the chapter addressing aggregate queries. In the next chapter, we followed Suciu's [Suc24] categorisation of the various approaches modelling the difference operation. We focused on two papers: one by Geerts and Poggi [GP10] which utilises the natural order of a semiring to define a monus operation, and another by Grädel and Tannen [GT25] which develops a semiring semantics for first-order logic.

Due to the limited scope of this work, certain topics were not developed. Most notably, recursive queries were not considered in this investigation. Green et al. [Gre+07] extended the semiring framework to recursive queries by providing proof-theoretic and fixpoint semiring semantics for Datalog. Building on this, [Deu+14] shows that Boolean circuits can compactly represent Datalog provenance, while [Dan+21] further generalises the framework to least fixed-point logic, based on Grädel and Tannen's work on provenance for first-order logic.

Additionally, the work of Suciu [Suc24] regarding semiring ideals was considered. Although the ideal-based construction provides a sophisticated treatment of difference, this was ultimately not included as the link to provenance is not yet explicitly articulated.

Notwithstanding recent progress, several important challenges remain within the field. Central to these is the development of a unified framework capable of expressing aggregate, recursive, and negative queries. While such a synthesis has yet to be fully realised, recent contributions offer promising directions. [Sen+25] presents ProvSQL, a PostgreSQL extension that tracks provenance for the relational algebra and aggregation, while [Abo+24] introduces an extension of Datalog to S-relations that encompasses both recursion and aggregation.

A persistent challenge lies in the trade-off between the generality of a provenance representation and its practical computational feasibility. This is best illustrated by the role of the provenance semiring $\mathbb{N}[X]$. As the free commutative semiring, $\mathbb{N}[X]$ provides the most general representation. This universality is theoretically indispensable, as it allows a single symbolic calculation to serve a broad spectrum of applications.

However, this high degree of generality comes at a substantial practical cost. The storage requirements for these polynomials, and the computational overhead required to manipulate them, incur at least a polynomial cost in both time and space, thereby imposing significant performance constraints on large data environments. Therefore, a key objective of research is to identify a balance between the generality and practicality of different provenance representations [Gla21].

In a similar vein, we claim that commutative semirings are not sufficiently general. While their axioms ensure compatibility with the relational algebra, the properties of commutativity and associativity result in a significant loss of structural provenance information as they obscure the sequence of operations (as evidenced by Query 1.0.1 on the *Edges* relation in Figure 1). To realise a richer provenance framework, research must look to algebraic structures capable of documenting the full history of a data transformation.

Bibliography

[Abi+95] Serge Abiteboul, Richard Hull, and Victor Vianu. *Foundations of Databases*. Addison-Wesley, 1995. ISBN: 9780201537710.

[Abo+24] Mahmoud Abo Khamis, Hung Q. Ngo, Reinhard Pichler, Dan Suciu, and Yisu Remy Wang. "Convergence of datalog over (Pre-) Semirings". In: *Journal of the ACM* 71.2 (2024). ISSN: 0004-5411. DOI: 10.1145/3643027.

[Ame84] Khaled Amer. "Equationally complete classes of commutative monoids with monus". In: *Algebra universalis* 18.1 (1984), pp. 129–131. ISSN: 1420-8911. DOI: 10.1007/BF01182254.

[Ams+11a] Yael Amsterdamer, Daniel Deutch, and Val Tannen. "On the Limitations of Provenance for Queries with Difference". In: *3rd USENIX Workshop on the Theory and Practice of Provenance (TaPP 11)*. 2011. DOI: 10.48550/arXiv.1105.2255.

[Ams+11b] Yael Amsterdamer, Daniel Deutch, and Val Tannen. "Provenance for aggregate queries". In: *Proceedings of the Thirtieth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*. PODS '11. Association for Computing Machinery, 2011, pp. 153–164. ISBN: 9781450306607. DOI: 10.1145/1989284.1989302.

[Bad+25] Guillermo Badia, Phokion G. Kolaitis, and Carles Noguera. "Codd's Theorem for Databases over Semirings". In: *Proceedings of the ACM on Management of Data* 3.5 (2025), pp. 1–26. DOI: 10.1145/3767713.

[Ben+08] Omar Benjelloun, Anish Das Sarma, Alon Halevy, Martin Theobald, and Jennifer Widom. "Databases with uncertainty and lineage". In: *The VLDB Journal* 17.2 (2008), pp. 243–264. ISSN: 0949-877X. DOI: 10.1007/s00778-007-0080-z.

[Bun+01] Peter Buneman, Sanjeev Khanna, and Tan Wang-Chiew. "Why and Where: A Characterization of Data Provenance". In: *Database Theory — ICDT 2001*. Ed. by Jan Van den Bussche and Victor Vianu. Springer, 2001, pp. 316–330. ISBN: 9783540445036. DOI: 10.1007/3-540-44503-X_20.

[CH85] Ashok K. Chandra and David Harel. "Horn clause queries and generalizations". In: *The Journal of Logic Programming* 2.1 (1985), pp. 1–15. ISSN: 0743-1066. DOI: 10.1016/0743-1066(85)90002-0.

[Che+09] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. "Provenance in Databases: Why, How, and Where". In: *Foundations and Trends in Databases* 1.4 (2009), pp. 379–474. ISSN: 1931-7883. DOI: 10.1561/1900000006.

[Cod70] Edgar F. Codd. "A relational model of data for large shared data banks". In: *Commun. ACM* 13.6 (1970), pp. 377–387. ISSN: 0001-0782. DOI: 10.1145/362384.362685.

[Cod71] Edgar F. Codd. "Normalized data base structure: a brief tutorial". In: *Proceedings of the 1971 ACM SIGFIDET (Now SIGMOD) Workshop on Data Description, Access and Control*. SIGFIDET '71. Association for Computing Machinery, 1971, pp. 1–17. ISBN: 9781450373005. DOI: 10.1145/1734714.1734716.

[Cod72] Edgar F. Codd. "Relational Completeness of Data Base Sublanguages". In: *Data Base Systems, Courant Computer Science Symposia Series, Vol. 6*. Prentice-Hall, 1972, pp. 65–98.

[Cui+00] Yingwei Cui, Jennifer Widom, and Janet L. Wiener. "Tracing the lineage of view data in a warehousing environment". In: *ACM Transactions on*

Database Systems (TODS) 25.2 (2000), pp. 179–227. ISSN: 0362-5915. DOI: 10.1145/357775.357777.

[Dan+21] Katrin M. Dannert, Erich Grädel, Matthias Naaf, and Val Tannen. "Semiring Provenance for Fixed-Point Logic". In: *29th EACSL Annual Conference on Computer Science Logic (CSL 2021)*. Ed. by Christel Baier and Jean Goubault-Larrecq. Vol. 183. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, 17:1–17:22. ISBN: 9783959771757. DOI: 10.4230/LIPIcs.CSL.2021.17.

[Deu+14] Daniel Deutch, Tova Milo, Sudeepa Roy, and Val Tannen. "Circuits for Datalog Provenance". In: *Proc. 17th International Conference on Database Theory (ICDT), Athens, Greece, March 24-28, 2014*. Ed. by Nicole Schweikardt, Vassilis Christophides, and Vincent Leroy. OpenProceedings.org, 2014, pp. 201–212. DOI: 10.5441/002/ICDT.2014.22.

[Gla21] Boris Glavic. "Data Provenance". In: *Foundations and Trends in Databases* 9.3–4 (2021), pp. 209–41. ISSN: 1931-7883. DOI: 10.1561/1900000068.

[Gol99] Jonathan S. Golan. *Semirings and Their Applications*. Kluwer Academic Publishers, 1999. ISBN: 9780792357865.

[GP10] Floris Geerts and Antonella Poggi. "On database query languages for K-relations". In: *Journal of Applied Logic* 8.2 (2010). Selected papers from the Logic in Databases Workshop 2008, pp. 173–185. ISSN: 1570-8683. DOI: 10.1016/j.jal.2009.09.001.

[Gre+07] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. "Provenance semirings". In: *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*. PODS '07. Association for Computing Machinery, 2007, pp. 31–40. ISBN: 9781595936851. DOI: 10.1145/1265530.1265535.

[Gre11] Todd J. Green. "Containment of Conjunctive Queries on Annotated Relations". In: *Theory of Computing Systems* 49.2 (2011), pp. 429–459. ISSN: 1433-0490. DOI: 10.1007/s00224-011-9327-6.

[GT17] Todd J. Green and Val Tannen. "The Semiring Framework for Database Provenance". In: *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. PODS '17. Association for Computing Machinery, 2017, pp. 93–99. ISBN: 9781450341981. DOI: 10.1145/3034786.3056125.

[GT25] Erich Grädel and Val Tannen. "Provenance Analysis and Semiring Semantics for First-Order Logic". In: *Model Theory, Computer Science, and Graph Polynomials: Festschrift in Honor of Johann A. Makowsky*. Ed. by Klaus Meer, Alexander Rabinovich, Elena Ravve, and Andrés Villaveces. Springer Nature, 2025, pp. 351–401. ISBN: 9783031863196. DOI: 10.1007/978-3-031-86319-6_21.

[IL84] Tomasz Imieliński and Witold Lipski. "Incomplete Information in Relational Databases". In: *Journal of the ACM* 31.4 (1984), pp. 761–791. ISSN: 0004-5411. DOI: 10.1145/1634.1886.

[Kar+10] Grigoris Karvounarakis, Zachary G. Ives, and Val Tannen. "Querying data provenance". In: *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*. SIGMOD '10. Association for Computing Machinery, 2010, pp. 951–962. ISBN: 9781450300322. DOI: 10.1145/1807167.1807269.

[KG12] Grigoris Karvounarakis and Todd J. Green. "Semiring-annotated data: queries and provenance?" In: *SIGMOD Record* 41.3 (2012), pp. 5–14. ISSN: 0163-5808. DOI: 10.1145/2380776.2380778.

[Klu82] Anthony Klug. "Equivalence of Relational Algebra and Relational Calculus Query Languages Having Aggregate Functions". In: *Journal of the ACM* 29.3 (1982), pp. 699–717. ISSN: 0004-5411. DOI: 10.1145/322326.322332.

[Lan02] Serge Lang. *Algebra*. Revised 3rd. Springer, 2002. ISBN: 9780387953854.

[Rab89] Lawrence R. Rabiner. “A tutorial on hidden Markov models and selected applications in speech recognition”. In: *Proceedings of the IEEE* 77.2 (1989), pp. 257–286. DOI: 10.1109/5.18626.

[Ros08] Benjamin Rossman. “Homomorphism preservation theorems”. In: *Journal of the ACM* 55.3 (2008). ISSN: 0004-5411. DOI: 10.1145/1379759.1379763.

[Sen+25] Aryak Sen, Silviu Maniu, and Pierre Senellart. “ProvSQL: A General System for Keeping Track of the Provenance and Probability of Data”. In: (2025). DOI: 10.48550/arXiv.2504.12058.

[Suc24] Dan Suciu. “Different Differences in Semirings”. In: *The Provenance of Elegance in Computation - Essays Dedicated to Val Tannen*. Ed. by Antoine Amarilli and Alin Deutsch. Vol. 119. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, 10:1–10:20. ISBN: 9783959773201. DOI: 10.4230/OASIcs.Tannen.10.